SECP3843 - PPG CASE STUDY

RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM) TECHNICAL REPORT

LIM YU HAN

LING YU QIAN

NAWWARAH AUNI BINTI NAZRUDIN

NG YU HIN

AHMAD ADIB ZIKRI BIN A.MAZLAM

NEO LI XIN

UNIVERSITI TEKNOLOGI MALAYSIA

ABSTRACT

For global manufacturing companies like PPG, with raw materials like volatile solvents and specialty pigments that have highly specific shelf-life requirements, effective inventory risk management is paramount. The aim of this technical report is to design, develop and implement an end-to-end analytics pipeline to solve two primary inventory risks: Recoverable Assets (RA)—MATERIALS with stock levels above the 12-months of projected material usage—AND Magna Carta (MC) risk—MATERIALS that have been inactive for 9 months or have expired and whose value needs to be completely written off.

The current PPG inventory management system leverages isolated and siloed data in six operational areas: materials, snapshots of inventory, purchase orders, production orders, supplier information, and sales orders, creating problems with data latency, manual calculations, and a reactive approach to risk management. These constraints were addressed by building a three-tier Medallion Architecture with the Bronze, Silver and Gold tiers hosted on the Microsoft Azure cloud platform. These raw operational data are ingested and stored into Azure Data Lake Storage Gen2 using Azure Data Factory, progressively cleansed and transformed using PySpark notebooks in Azure Databricks and then structured into a star schema dimensional model in Azure Synapse Analytics.

The Gold layer ships the fact and dimension tables that are curated as server-less SQL views and are linked to an interactive Microsoft Power BI dashboard. The dashboard allows to monitor the health of inventory in real-time on various risk aspects such as RA classification, identification of MC dead stock and expired stock, stock out, estimation of financial provision, and mapping of material shortages to affected customer sales order. Dynamic risk calculations and KPI visualisation are supported by custom DAX measures.

System testing over three pipeline layers (Bronze, Silver and Gold), Azure Synapse Analytics integration and User Acceptance Testing (UAT) verified successful identification of RA

inventory, MC dead stock, MC expired stock, stockout risks, and financial provisions. The pipeline passed all test cases, demonstrating its ability to reliably convert operational data from multiple sources into a coherent, scalable, and actionable analytics platform. The solution shows how Cloud native data engineering technologies can transform.

Keywords: Inventory Risk Management, Recoverable Assets, Magna Carta Risk, Medallion Architecture, Microsoft Azure, Azure Data Factory, Azure Databricks, Azure Synapse Analytics, Power BI, Supply Chain Analytics, Data Engineering.

TABLE OF CONTENTS

LIST OF FIGURES

FIGURE NO. **TITLE** **PAGE**

LIST OF TABLES

FIGURE NO. **TITLE** **PAGE**

CHAPTER 1

INTRODUCTION

1.1 Overview

The capacity to have lean and responsive supply chain in the contemporary industrial environment has become one of the key determinants of the financial wellbeing of a company. In the case of global manufacturing giants such as PPG, where the inventory of raw materials at the facility includes not only volatile solvents but also specialized pigments, a more sophisticated and integrated approach to information is required and not only through traditional bookkeeping. As Reis and Housley (2022) define it, switching to the automated data system enables organizations to leave the stage of mere data collection and transition to the stage of generating high-trust insights that lead to executive decision-making.

In order to solve the issue of inventory obsolescence, the system must redesigned through the creation of an end-to-end analytics pipeline on the Microsoft Azure platform. It separates and purifies raw operational data into structured format to be used by a high-level business intelligence by deploying a multi-layered Medallion Architecture. Instead of using the traditional, manual reports, the solution is based on the identification of Recoverable Assets (RA) and Magna Carta (MC) risks. The end result is to have a visualization of the inventory health in real-time so that the organization is able to reduce financial waste and at the same time production requirements with regard to inventory are met without disruption.

1.2 Problem Background

The handling of chemical and industrial raw materials is a risky affair since products like pigments, resins have a short shelf-life, which is usually rigid. According to Silver, Pyke, and Thomas (2016), the first inefficiency in the supply chain is the phenomenon of frozen capital, meaning that the company finances are frozen in the untapped inventory. It is found that a major visibility gap is currently experienced by PPG due to the fact that the inventory snapshots, production orders and sales information are currently stored in fragmented data silos. This is not integrated and as such, it is hardly possible to detect at-risk materials until it is too late when it is expired or the so-called dead stock.

This paper is focused on two risks. To start with, the availability of RA which are resources that have amounts that are beyond 12 months of the estimated usage. It is thought that non-detection of such excesses at the early stages leads to soaring cost of warehousing and high insurance premiums. Second, MC risk is a serious threat as far as finances are concerned; it is the materials whose consumption has not occurred at all in nine months or whose expiry dates have already elapsed. In professional accounting, these materials demand a 100% financial write-down as this directly decreases the net profit of the company.

Additionally, the lack of connection between inventory and sales records implies that in case there is a shortage of the materials, the company cannot know instantly which particular customer sales orders are to be postponed. These result in reactive firefighting culture in which relationships with clients are ruined because of unpredictable lead times. Therefore, it is urgently required to have a single system that would help fill these gaps and convert disjointed files into an active risk-management tool.

1.3 Problem Statement

Although raw operational data are available, the absence of an automated and unified analytical system at PPG cause problems as follows:

- **Inventory obsolescence**: Raw materials that have a limited shelf time usually go out of order before they can be used up, causing losses.
- **MC risk**: The materials that has not been used within a period of nine months or so will be written off financially by 100 percent, which in turn will affect the profitability.
- **Information silos**: There are six distinct datasets (materials, inventory, purchase orders, production orders, suppliers, sales orders) which can hardly be compared manually.
- **Reactive Management**: The traditional systems see the issues only when they are too late and they must respond to the shortages or excess inventory.
- **Inability to scale**: Manual processes are susceptible to errors and cannot be expanded to support the modern supply chain demands.

1.4 Project Objective

The following are quantifiable objectives that guide this project:

- To research and examine RA and MC business logic in the PPG scenario.
- To create and build a three-tier Medallion Architecture (Bronze, Silver, Gold) on Microsoft Azure.
- To have a Multidimensional Data Model that is able to compute the total amounts of financial provisions.
- To view the visualization of inventory risks and affected sales orders with an interactive Power BI dashboard.

1.5 Project Scope

The project focused on six synthetic datasets that represent core supply chain operations which are:

- materials.csv
- inventory_snapshot.csv
- purchase_orders.csv
- production_orders.csv
- suppliers.csv
- sales_orders.csv

These datasets represent the PPG raw data and will be integrated. It is for enabling the cross-functional analysis of inventory behavior and demand. Furthermore, to create and build the three-tier Medallion Architecture on Microsoft Azure, by using it, Microsoft Azure as the main platform. The utilization of Azure Blob Storage is for the storage of raw and processed data. Meanwhile the process of data ingestion and orchestration is used by Azure Data Factory before being delivered to Power BI for visualization and reporting.

Plus, the system is designed to support some functionalities:

- Risk Detection
    - Identification of RA (excess inventory > 12 months)
    - Identification of MC risks (inactive or expired inventory)

- Inventory Monitoring
    - Detection of excess stock
    - Detection of expired materials
    - Detection of dead stock

- Financial Impact Analysis

    - Estimation of inventory holding costs

- Sales & Operations Impact
    - Identification of stockouts
    - Mapping shortages to affected customer sales orders

- Visualization
    - Development of an interactive dashboard to monitor inventory health in real time

1.6 Project Importance

To help reduce financial losses and improve operational efficiency in supply management, PPG should be alert by detecting RA and MC risk early. This can minimize inventory write-offs, reduce excess stock, and avoid frozen capital tied up in unused materials. The system transforms fragmented data into a unified analytics platform which allows management to monitor inventory health in real time and make faster decisions. This will reduce reliance on manual reporting that will lead to less human error that will improve overall efficiency in inventory planning and control.

The change from reactive problem-solving to proactive decision-making put risk management at its top. By integrating inventory, production, and sales data, the system helps ensure materials are available when needed. No overstocking and stockouts occur. The use of cloud technologies such as Microsoft Azure, Azure Data Factory, and Power BI also makes the system scalable and future-ready. This allow companies to expand their analytics capabilities.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Modern supply chains generate large amounts of data from inventory, production, and sales activities. Without a structured data architecture, these datasets remain fragmented and difficult to analyze. It will limit an organization's ability to detect inventory risks such as excess stock, dead stock, and stockouts. In the context of the PPG supply chain scenario, the transition from manual spreadsheet-based tracking to an automated analytics pipeline requires a strong theoretical foundation in three aspects. They are inventory management logic, financial provisioning principles, relational data modeling, and modern cloud data architecture. In addition, the review explores Medallion Data Architecture, cloud computing platforms such as Microsoft Azure, and modern data pipeline approaches such as ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform). Together, these theories and technologies form the foundation for the design and implementation of the inventory risk analytics system proposed in this study.

2.2 Supply Chain Inventory Management

In the context of PPG's supply chain, a solid understanding of inventory logic, accounting principles, and relational data modeling is essential for the shift from manual inventory management to automated data engineering.

2.2.1 Supply Chain Management in Inventory

Supply Chain Management (SCM) is the necessary structure to support good inventory management, especially in a complicated manufacturing setting such as PPG. It is the coordinated and integrated management of material, information and financial flows from initial suppliers to production, and from production to final customers in the complete supply chain. The main objective is to make sure that the right materials are provided in the right amount, at the right time and at the lowest cost, in order to optimize the way of working.Basic concepts of SCM in inventory.

A fundamental principle of SCM is achieving an optimal balance between supply and demand.

- Excess Inventory: Leads to significant carrying costs (storage, insurance, capital lock-up).
- Insufficient Inventory: Causes costly stockouts, production delays, and failure to meet customer demand.
  The SCM approach seeks to find the level that minimizes financial burden while maintaining seamless operations.

SCM is critical for integrating disparate operational datasets, transforming fragmented data into a cohesive system. This integration connects various stages of the supply chain:

- Upstream: Suppliers and Purchase Orders
- Internal Operations: Inventory Snapshots and Production Orders
- Downstream: Sales Orders (Customer Demand)

These elements are linked together by SCM to create a full view of the material flow lifecycle, enabling organizations to see the material flow from beginning to end. This is important to be able to spot potential inefficiencies like over-stocking, under-stocking or slow-moving inventory. The tracking of inventory, from its origin to its destination.Tracking of stock, from source to destination.

Without a comprehensive SCM system, there are data silos that result in reactive management and inaccurate tracking. As mentioned in this project, a comprehensive data engineering pipeline can be put in place to centralize scattered data. This change will allow for better real-time tracking and help to make informed decisions in advance by giving visibility into material use and demand.

SCM attaches high importance to inventory risk management, particularly for materials that are time sensitive or perishable. In relation to the PPG context, some of the important inventory risks that have been identified and managed are:

- Excess Inventory (Recoverable Assets or RA): Materials exceeding 12 months of expected usage.
- Obsolete/Inactive Inventory (Magna Carta or MC): Materials with no usage over 9 months or those that have expired.

The early identification of these risks, which directly affect financial and operational performance, can enable remedy measures, including reallocation, demand adjustment and liquidation measures.

In a nutshell, Supply Chain Management is the basic framework that combines all the inventory information within the business functions. It helps to make a critical transition from scattered, reactive inventory to data-driven proactive inventory management. This modern approach is vital to minimize waste, improve service levels and, ultimately, supply chain performance.

2.2.2 Inventory Management Logic

Modern inventory management is about the concept of "lean" operations, where having the right amount of inventory on hand to satisfy demand is balanced with reducing the capital tied up in inventory. Under the PPG concept, this is addressed by implementing the RA principle.

- A key rule for identifying RA is the 12-Month Consumption Rule: Any inventory quantity that surpasses the forecasted consumption for the next year is categorized as "Excess Stock."

- The Operational Impact of identifying RA is significant. Flagging these assets enables planners to intervene before materials expire. This triggers "recovery" actions, such as reallocating materials to other production lines or offering discounts for a quick sale. As noted by Silver et al. (2016), reducing this excess inventory is crucial for lowering "carrying costs," which encompass expenses like warehousing, insurance, and the opportunity cost associated with frozen capital.

2.2.3 Financial Provisioning

Even though excess stock are an annoyance, "Dead Stock" is a direct blow to the balance sheet. In the chemical sector, there are raw materials such as resins and pigments that are volatile, and will have their own specific shelf life.

- **The MC Definition**: This is a high-risk category for materials that have shown zero consumption for 9 consecutive months or have already reached their expiration date.

- **100% Financial Write-down**: Under standard accounting "provisions," once a material hits the MC threshold, it is no longer considered an asset. The company must record a 100% financial write-down. This means the value of that inventory is effectively "erased" from the profit margins, acting as a total loss.

- **The Necessity of Early Warning**: Data engineering pipelines aim to transform this from a "post-mortem" realization into a predictive warning, allowing the company to avoid the "MC" death sentence through proactive inventory aging analysis.

2.3 Data Architecture

The Medallion Architecture is a data design pattern that organizes data into three progressive tiers of quality and structure to enable organizations to build reliable and scalable analytics systems. This architecture, originally formalized by Databricks (n.d.), establishes a clear separation between raw ingestion, validated transformation, and curated presentation of data. The purpose is to organize unstructured data lakes. It is also commonly known as "multi-hop" architecture. Within the context of the PPG inventory risk analytics project, the Medallion Architecture serves as the foundational framework for converting six fragmented operational CSV files into a governed dataset that supports the detection of RA and MC risks.

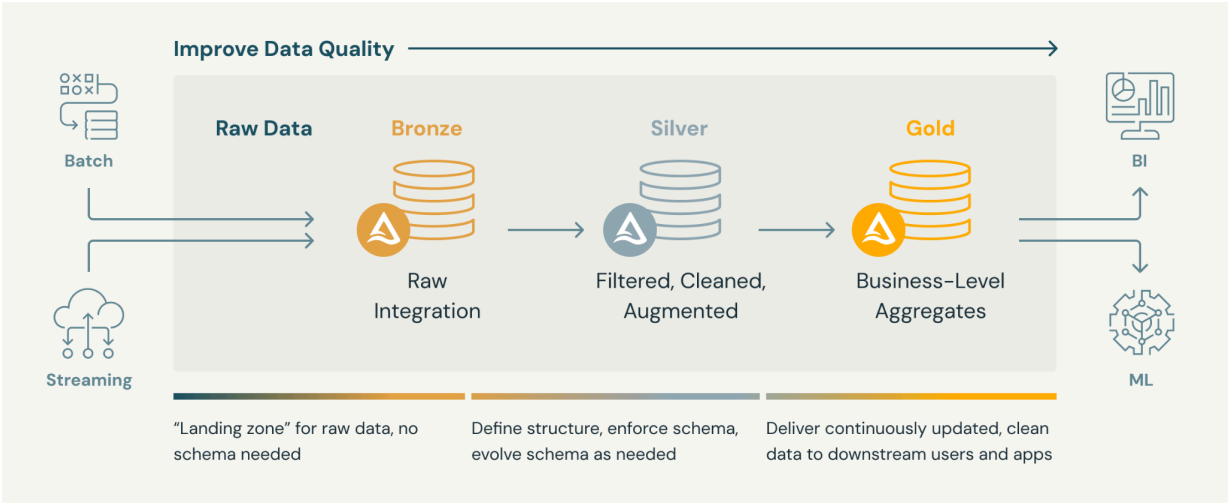


Figure 2.1 The Medallion Architecture (Databricks, n.d.)

As illustrated in Figure 2.1, the Medallion approach consists of three main layers:

- **Bronze**: This layer serves as the initial landing zone for storing raw data from all source files. The data is ingested in its rawest form to preserve the original state and provide a complete audit trail. The defining characteristic of the Bronze layer is immutability because data stored in this tier is not altered or deleted. This supports the concept of a source of truth, as any errors in later stages can be traced back to the original data and reprocessed (Reis & Housley, 2022). The raw data remains available for recovery when data quality issues are found.

- **Silver**: The Silver layer functions as the validation engine where the raw data is cleaned and normalized. This tier represents the transformation phase of the ELT (Extract, Load, Transform) paradigm, in which data is first loaded into the data lake before business logic is applied. Errors are corrected, and missing values are handled. Duplicate records are removed to improve data quality. The datasets are then combined to form a consistent structure.

- **Gold**: The Gold layer represents the final stage in the pipeline, where the data is ready for business use. At this tier, Silver data is restructured into a Fact Constellation

schema. It organizes data into a central fact table and multiple dimension tables, which makes it easier to aggregate and analyze using reporting and visualization tools such as Power BI. Specific logics are applied here to aggregate the refined data into actionable metrics for detecting RA and MC risks.

The Medallion Architecture improves data quality by enforcing validation at each layer before data advances further in the pipeline. Each stage of the architecture represents a new data accuracy and quality challenge. Data teams must ensure that issues such as missing values, schema mismatches, and duplicate records are detected and corrected before reaching the final analytical layer (Bergh, 2024).

2.4 Data Modeling

To turn fragmented CSV files into actionable insights, the data must be structured for analytical performance rather than just storage. Dimensional modeling, popularized by Kimball and Ross (2013), provides the foundational framework for this project. While a traditional Star Schema positions a single Fact Table at the center containing quantitative metrics surrounded by descriptive Dimension Tables, complex supply chains require a more advanced variation known as a Fact Constellation Schema. In this model, multiple fact tables share conformed dimension tables, allowing different business processes to be analyzed concurrently as shown in Figure 2.2.

This approach offers several key advantages tailored to the PPG inventory ecosystem. In terms of performance, it minimizes complex, resource-heavy joins, allowing tools like Microsoft Power BI to aggregate millions of rows efficiently. From a usability perspective, shared dimensions provide an intuitive structure for business users; for example, it makes it easier to understand how a stockout tracked in the inventory fact table directly impacts pending fulfillments in the sales order fact table via a shared "Material Dimension." Additionally, the Fact Constellation Schema ensures consistency by reducing data redundancy commonly found in flat files, so that updates such as changes to a supplier’s metadata are reflected uniformly across the entire analytical dataset.

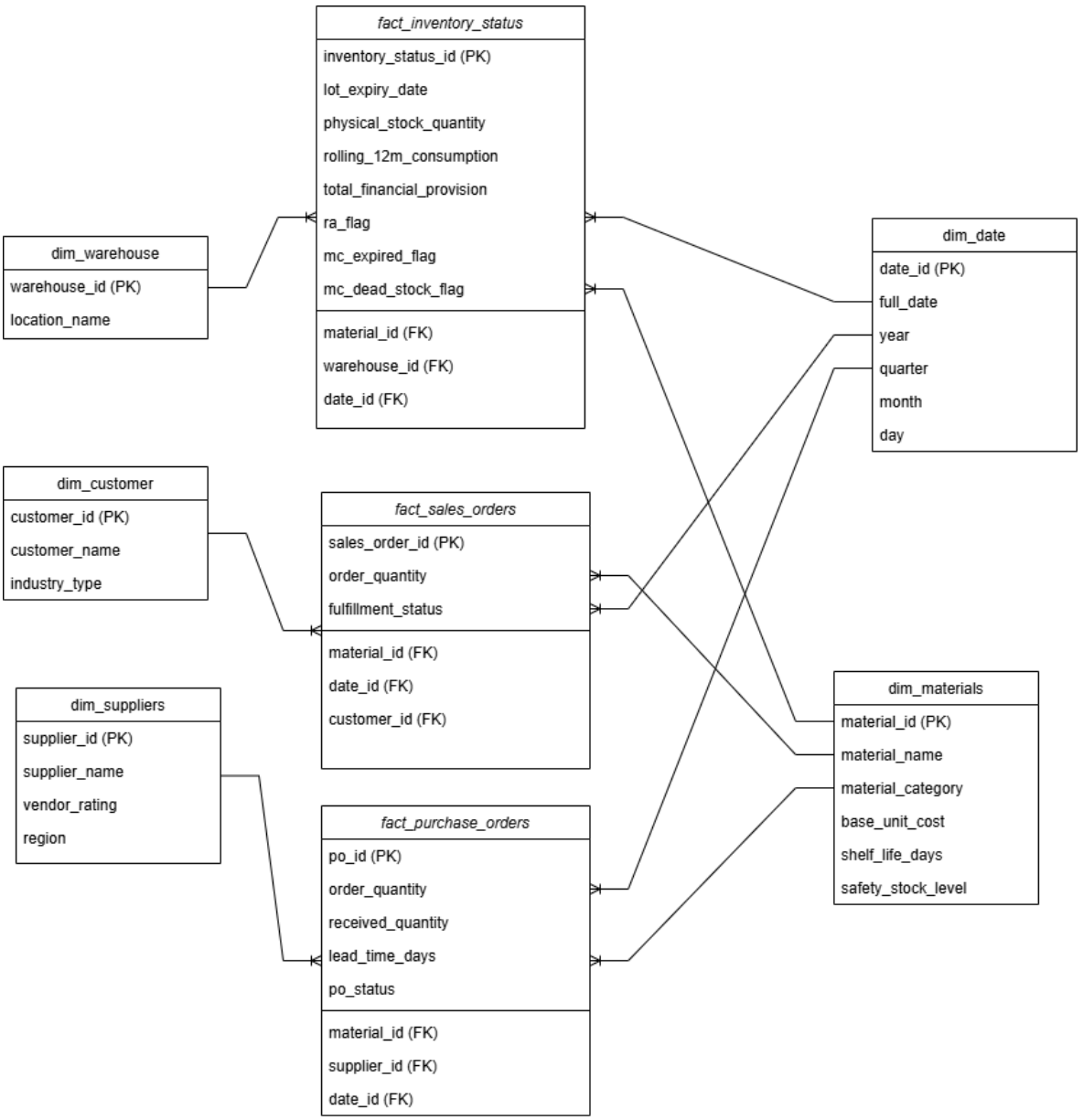


Figure 2.2. Facts Constellation Schema with Fact Tables and Dimension Tables

2.5 Enterprise Resource Planning (ERP)

As we know, Enterprise Resource Planning (ERP) refers to the technology used to integrate core business processes (Staff, 2026b). It gathers data from across the organization into one central database. That is why it is called “single source of truth”. In the PPG context,

the ERP is really needed for ensuring data consistency and operational coordination. All of the business transactions are recorded through ERP in real time.

However, ERP is not designed for advanced analytics. Relying only on ERP can cause issues such as excess inventory or dead stock. So, to overcome it, a data pipeline is required so that it can extract, transform, and load (ETL/ELT) this raw data into a structured format suitable for analysis. This pipeline enables the application of inventory management logic, such as the identification of excess (RA) and obsolete (MC) stock, ultimately supporting data-driven decision-making.

2.6 Technology Comparison

Selecting the most suitable technology is crucial as it provides most compatible services to manage complex inventory data and detect RA and MC risks automatically. This section will compare the traditional data management approach against modern cloud-based approach to justify the selected technology.

2.6.1 Cloud vs. On-Premise

The scalability and cost-efficiency are the factors that differentiates Microsoft Azure from traditional local SQL servers. On-premise SQL servers need an upfront capital expenditure (CapEx) for hardware, licensing and ongoing maintenance (Erl et al., 2013). In addition, organisations must over-provision hardware to support peak processing loads which leaves costly hardware as excess capacity during off-peak times. Cloud platforms like Microsoft Azure, on the other hand, offer dynamic elasticity and a more flexible operational expenditure (OpEx) model. It is immensely useful for inventory risk management systems because the required computing resources can be adjusted to accommodate the changing volume of data. This elasticity means that organisations only pay for services used, without the cost burden and inflexibility of on-premise solutions.

2.6.2 ETL vs. ELT

Traditional data pipeline would usually involve ETL pipeline where data is extracted, transformed in a separate server then loaded into the data warehouse. While it does work great for smaller datasets, that is not the case when massive data volumes come into play in modern-day data engineering. Therefore ELT pipelines like the ones represented in Figure 2.3 are desirable as they address the issue of transformation bottlenecks by leveraging the enormous processing capabilities of cloud data warehouses and data processing tools (Reis & Housley, 2022).

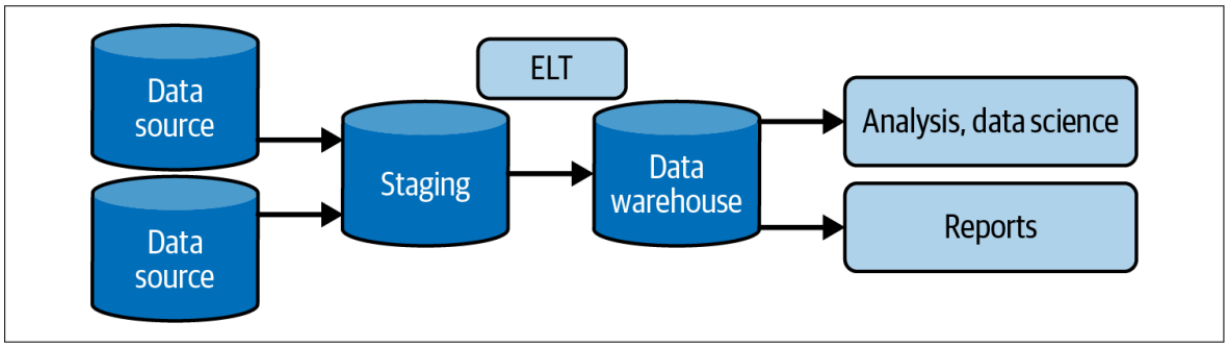


Figure 2.3. ELT Process Flow (Reis & Housley, 2022)

For the purpose of this case study, the raw CSV files will be extracted and loaded into a scalable Data Lake (the Bronze layer). They are then within cloud computing engines to form complex inventory metrics, specifically for identifying RA and MC risks. The end result of this overall ELT pipeline not only reduces processing bottlenecks when handling massive ERP CSV exports, but also maintains a historical catalogue of the raw inventory data for future finance reporting.

2.6.3 Visualization Tools

According to the traditional design, the inventory reports are prepared based on individual Microsoft Excel spreadsheets that have issues resulting from human errors when processing millions of rows of data. Modern solutions use data pipelines to feed data into data visualization tools like Microsoft Power BI. Power BI uses a compact in-memory engine with the Data Analysis Expressions (DAX) language to analyze large data sets that have complex relational database models (Microsoft, 2023). The system will use Microsoft Power BI and the DAX language to automate the inventory reporting process for RA and MC inventory. This new improvement will automate calculations that would otherwise be complicated for team members to manually calculate as well as create interactive data visualizations for stakeholders.

2.7 Related Work and Case Studies

In the manufacturing industry, data engineering pipelines have successfully reduced Days of Inventory on Hand (DIOH) through the integration of real-time data from supply chains, production systems, and inventory systems (BPCS, 2025). With the help of Azure-powered Inventory Control Towers and ELT pipelines, a global manufacturer reduced inventory by over 15%, improved forecasting accuracy by 20%, and saved over $100 million annually by optimizing stock levels and minimizing excess holdings (BPCS, 2025). With real-time insights from integrated data pipelines, WNS' analytics-led model for a manufacturing major increased on-time-in-full (OTIF) deliveries by 5-10% and cut carrying costs (WNS, 2025).

Cloud-native tools like Microsoft Azure offer a proven industry approach. They handle complex data transformations and scalable ETL processing (Nalluru, 2024). Manufacturing enterprises like FRÄNKISCHE Industrial Pipes and Epiroc demonstrate this success. They use Azure Data Factory and cloud architecture to integrate scattered data. This process creates a unified view from hybrid sources.

Official Microsoft case studies highlight Epiroc's specific achievements. Epiroc established an AI and data factory on Azure. This platform standardized their operational processes globally. It significantly increased efficiency and reduced product waste (Microsoft, 2023). These companies actively moved away from manual coding. They leveraged Azure's scalable environment instead. This shift eliminated the latency found in traditional data integration methods.

The transition enables real-time, cloud-based data processing. It allows for the proactive management of inventory and stockout risks. Consequently, supply chain planners can reallocate RA before they expire.

Using the Medallion Architecture (Bronze-Silver-Gold layers on Azure Data Factory), these implementations ingest multi-source CSVs, apply RA/MC logic, and deliver a fact constellation schema for Power BI dashboards addressing DIOH directly through excess stock detection (RA: >12 months consumption) and obsolescence flagging (MC: 9+ months no use or expired) (Databricks, 2022; Bergh, 2024). With real-time insights from integrated data pipelines, ETL/ELT workflows support financial provisioning and sales order impact analysis through ERP data aggregation for demand forecasting (WNS, 2025; Reis & Housley, 2022).

2.8 Comparison with Previous Studies

To validate the proposed data engineering solution for PPG's inventory risk management, it is essential to compare the planned architecture against the established industry implementations reviewed in Section 2.5. Rather than deviating from industry norms, the proposed pipeline deliberately adopts the proven frameworks utilized by global manufacturing enterprises like Epiroc, BPCS, and WNS.

The following table summarizes how the proposed PPG project aligns with and implements the successful strategies identified in previous industry case studies:

Architectural / Business Component	Industry Implementations (BPCS, WNS, Epiroc)	Proposed PPG Analytics Pipeline
Data Processing Framework	Utilizes scalable ELT pipelines within cloud environments to handle massive datasets efficiently, often processing live ERP streams from hybrid on-premise and cloud sources (BPCS, 2025; WNS, 2025).	Extracts six synthetic CSV files into Azure Data Lake Storage Gen2 (Bronze), then applies ELT transformations using Azure Databricks. Scope is limited to batch CSV exports rather than live ERP streams, which is appropriate for a proof-of-concept context.
Architectural Design	Implements the full Medallion Architecture (Bronze, Silver, Gold) on Azure Data Factory, typically integrated with enterprise-grade monitoring and CI/CD pipelines (Databricks, 2022; Bergh, 2024).	Adopts the exact three-tier Medallion structure using Azure Data Factory for orchestration and Azure Databricks for processing.
Inventory Risk Logic	Applies RA (>12 months excess stock) and MC (9+ months inactivity or past expiry) logic to detect at-risk inventory and trigger financial write-downs (WNS, 2025; Databricks, 2022).	Automates the exact RA and MC business logic via Databricks to flag at-risk materials and compute 100% financial write-downs.

Visibility and Delivery	Deploys real-time Inventory Control Towers via Power BI for cross-functional, enterprise-wide supply chain visibility, often connecting to live data feeds (Microsoft, 2023; BPCS, 2025).	Delivers an interactive Power BI dashboard featuring DAX measures, KPI scorecards, and cross-filtering for sales order impacts.
Scale and Scope	Deployed globally across enterprise hybrid sources to achieve millions in cost savings, reduce product waste, and improve forecasting accuracy by 20% (BPCS, 2025; WNS, 2025).	Serves as a targeted, end-to-end proof of concept focused strictly on six core operational datasets to prove risk detection viability.

Table 2.1: Comparison of Proposed Pipeline with Previous Industry Studies

2.9 Microsoft Azure

The cloud platform selection influences the extent to which the source CSV files can successfully be converted to actionable insights. Microsoft Azure was chosen due to the fact that its services perfectly match the three layer Medallion Architecture. In contrast to generic cloud-based storage solutions, Azure offers a full ecosystem, where ingestion of raw data, transformation, and delivery to the dashboard can be managed under a single platform. This remove the intricacy of joining together services offered by various vendors. In addition, the project requires a platform capable of supporting batch processing of historical inventory snapshots as well as quick querying of the ultimate Power BI dashboard. Azure fulfills both requirements using its integrated service architecture (Rucco et al., 2024).

2.9.1 Map of Azure Services to PPG Pipeline Layers.

The Medallion Architecture is mapped to particular services of Azure. In the case of Bronze Layer, the six raw CSV files get stored in the Azure Data Lake Storage Gen2 as received, with the original data saved so that it can be audited (Carrilho, 2025). This imbalance is essential since in case any transformation logic requires reconfiguration, then the entire pipeline can be reprocessed using the original source of data. In the case of Silver layer, Azure Databricks uses the given PPG business logic. This involves the computation of 12-month rolling consumption of every material, the marking of materials whose current stocks are above the set limit as Recoverable Assets, and the recognition of Magna Carta materials. Azure Databricks can perform these window operations and date comparisons in a highly efficient manner on the entire dataset. In the case of Gold layer, the final Fact Constellation schema tables are stored in Azure SQL Database or Azure Synapse, then this structured format is directly linked to Power BI to visualise the dashboard.

2.9.2 Azure Data Factory to orchestrate Pipelines.

The project will demand the pipeline to execute in a synchronized order: ingest CSV files, run transformations, and load results into the Gold layer. This whole work process is coordinated by Azure Data Factory. In particular, Data Factory moves CSVs in their source locations to the Azure Data Lake Storage (Bronze), runs the Azure Databricks notebook with the RA and MC transformation code (Silver), and transfers transformed data to the Gold layers tables. This staging helps in making sure that the various layers are finished successfully before the next one gets started and also in case one step fails the pipeline can be restarted at the failure point as opposed to starting it up again.

2.9.3 Power BI Dashboard Delivery Integration

The end project deliverable will consist of a power BI dashboard to answer seven targeted questions such as what materials fall below the reorder levels and how customer sales

orders are affected by stockouts. With native integration to Power BI, this can be done with a lot of ease using Azure. After populating the Gold layer tables in the Azure SQL Database, power BI also connects to it in real-time with built-in connectors providing an incremental refresh. This implies that as new inventory data is fed, the dashboard is updated automatically. To the management team at PPG, this will mean real-time access to MC provisions and at-risk sales orders.

2.10 Microsoft Power BI

This project uses Microsoft Power BI, which is a self-service, unified business intelligence platform to develop the final analytical layer in inventory management. It is the main entry point of the stakeholders to the Gold layer of the Medallion Architecture.

2.10.1 Data Connectivity and Integration

The system employs the use of Microsoft Power BI to reach out to the data warehouse in Azure. Microsoft Power BI preserves the relational integrity achieved in the transformation stage by importing the Fact Constellation Schema which includes the fact-inventory-status table and related dimensions (Materials, Suppliers, Date, etc.) into the database.

2.10.2 Data Analysis Expressions (DAX)

The project uses Data Analysis Expressions (DAX) to go beyond mere data viewing. The complex RA and MC business logic is automated by creating custom measures using DAX. The essential measurements that are computed using DAX are:

- **Total Financial Provision**: A dynamic value of flagged material (MC Dead Stock or Expired) value.
- **Inventory Risk Flags**: Indicators that assess the differences between inventory levels and reorder levels to detect stockout risks.

- **RA Identification**: Determining the materials that are over the rolling 12-month consumption threshold.

2.10.3 Interactive Visualization and Reporting

The last dashboard is to be an Inventory Control Tower. It offers real-time inventory health visibility by a number of essential visual components:

- **KPI Scorecards**: The value of MC provisions and at-risk sales order numbers in real-time.
- **Cross-Filtering Capabilitie**s: This will enable the user to pick a certain at-risk material and immediately view what sales orders of customers will be affected.
- **Drill-down Functionality**: This allows supply chain planners to navigate between a high-level category view (e.g., Resin, Pigment) to batches or lot expiry dates.

2.11 Summary

In order to build an effective inventory risk analytics system, it is important to have a strong foundation in inventory management logic, financial provisioning principles, data modeling, and cloud data architecture. The concepts of RA and MC risk demonstrate how insufficient inventory and dead stock can significantly impact operational efficiency and financial performance. Early identification of insufficient stock helps reduce carrying costs such as warehousing, insurance, and opportunity cost. Meanwhile, the identification of dead stock ensures that financial write-downs are properly recognized and managed. The Fact Constellation schema model enables efficient analytical processing and supports business intelligence reporting. Microsoft Azure provides scalability, flexibility, and processing power compared to traditional on-premise systems, while ELT pipelines allow large datasets to be processed efficiently within cloud environments. Visualization tools such as Power BI further support decision-making. It is by transforming complex inventory data into interactive dashboards and actionable insights. In the end, industry implementations demonstrate that integrating cloud data pipelines, structured data models, and business intelligence tools can

significantly reduce inventory costs, improve forecasting accuracy, and enhance supply chain visibility.

CHAPTER 3

METHODOLOGY

3.1 Introduction

Systematic methodology is critical in converting business needs into a working data analytics system. In the absence of well-defined methodology, data engineering projects may deliver pipelines that are hard to maintain, not well-documented, or do not meet the expectations of stakeholders. In this chapter, the methodology that will be used in the implementation of the inventory risk management solution is introduced.

The project uses the System Development Life Cycle (SDLC) as the general model. SDLC offers a framework of steps to follow in a project, starting with its conception, all the way to the final deployment and maintenance. Dennis, Wixom, and Tegarden (2015) argue that SDLC is especially appropriate with information systems since it has defined deliverables, milestone review, and quality control checkpoints in every phase. These features are suitable to the needs of this project, which implies numerous data sources, particular business logic to classify the inventory, and a final dashboard deliverables.

One of the benefits of SDLC in this project is its ability to support parallel work streams in the implementation phase. Rucco et al. (2024) note that cloud-based data engineering initiatives can enjoy the benefits of parallel development since services like storage, compute and visualisation can be set up separately and then integrated.

3.2 The Chosen Methodology

The Waterfall model under the System Development Life Cycle (SDLC) framework is selected as the development methodology for the PPG recoverable assets and inventory risk management system. The structure of this model is illustrated in Figure 3.1, which presents a

linear sequence of development phases from requirement analysis through to maintenance (Simran, 2025).

The Waterfall model is suitable for this case study because the project scope, data sources, and analytical objectives are clearly defined at the initial stage. The inventory datasets, including materials, inventory, production orders, purchase orders, suppliers, and sales data, are already identified. Recoverable Assets (RA) and Magna Carta (MC) risk analysis as the expected outputs are also fixed. This reduces the need for iterative requirement changes and supports a structured development approach.

The system involves the integration of multiple data sources into a cloud-based environment using Microsoft Azure and a Medallion Architecture. As the data pipeline and transformation logic must be clearly defined before implementation, a sequential methodology ensures proper planning and reduces the risk of design inconsistencies (Simran, 2025).

Figure 3.1 reflects the disciplined flow required for this project, where each phase must be completed and validated before proceeding to the next (Simran, 2025). This is particularly important for ensuring the accuracy and reliability of financial and inventory risk outputs generated through the Power BI dashboard.

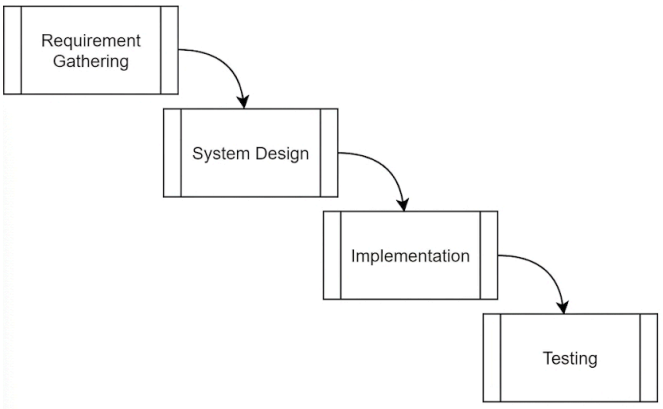


Figure 3.1 Waterfall Model

3.3 Phases of the Chosen Methodology

The Waterfall System Development Life Cycle is divided into several phases. Each phase produces a clear output that serves as input for the next phase. These methods help to make the requirements clear and to make sure the project's objectives are clear.

Phase 1: Requirement Analysis

This phase is concerned with studying the existing data environment in the context of PPG inventory management to gain insight into the current data management of raw materials, inventory, production and sales. The study analyzes several independent datasets such as materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders to look at the data movement between supply chain operations. This analysis reveals several critical problems, including data silos, lack of integration, slow reporting and manual analysis. The difficulties associated with these challenges make the identification of Recoverable Assets (RA) and Magna Carta (MC) risks challenging and time consuming.

To overcome these challenges and gather the system requirements effectively, the project utilized an analytical approach rather than traditional user interviews. The core activities conducted during this phase included:

- **Document Analysis and Business Logic Synthesis:** The team analyzed existing enterprise inventory tracking documents to establish the operational boundaries of inventory risks. Specifically, the definitions for Recoverable Assets (RA) and Magna Carta (MC) risks were extracted and translated into strict technical data transformation rules.
- **Dataset Schema Inspection:** A thorough investigation of the six core CSV files was performed. This activity allowed the team to map out the data ingestion requirements and identify crucial relational database elements such as primary keys and foreign keys to resolve the existing data silos.
- **Domain Benchmarking:** To determine the technical specifications required for the cloud architecture, the team analyzed industry-standard frameworks and documented case studies from global manufacturing enterprises. This benchmarking activity directly

informed the decision to utilize the three-tier Medallion Architecture on Microsoft Azure.

Based on these findings, high-level system requirements and project boundaries are defined to ensure the proposed data engineering solution addresses end-to-end inventory visibility and risk detection rather than isolated data processing tasks.

Phase 2: System Design

In this phase, the overall architecture of the inventory risk analytics system is designed based on the requirements identified earlier. To translate the high-level business needs into strict technical specifications, the team conducted the following core system design activities.

- **Cloud Architecture Blueprinting:** The team mapped out the physical data pipeline by designing a cloud-based Medallion Architecture. This activity involved explicitly assigning processes to the Bronze layer for raw data ingestion, the Silver layer for data cleaning and integration, and the Gold layer for analytical modeling and serving.
- **Dimensional Data Modeling:** The team designed the structural relationships between the disparate datasets such as inventory, production and sales, by developing a Fact Constellation Schema. This activity involved creating the Entity-Relationship Diagram (ERD) to define the specific fact tables, conformed dimensions, primary keys, and foreign keys required to ensure seamless data flow across the supply chain.
- **Transformation Logic Formulation:** Pipeline workflows were designed to execute the specific business calculations. The team formulated the step-by-step algorithmic logic needed to process raw data into the final metrics used for detecting Recoverable Assets (RA) and Magna Carta (MC) risks, as well as calculating financial provisioning.

Phase 3: Implementation

During the implementation phase, the designed architecture is developed into a functional data pipeline on the Microsoft Azure platform. Azure Blob Storage is used to store raw and processed data, while Azure Data Factory is utilized to orchestrate data ingestion and

transformation processes following the ELT approach. The six datasets are ingested into the Bronze layer, transformed and cleaned in the Silver layer, and structured into a Fact Constellation Schema in the Gold layer. Business logic for detecting RA and MC risks is implemented at this stage. In addition, Microsoft Power BI is used to develop interactive dashboards that visualize inventory health, financial impact, and supply chain risks. The implementation strictly follows the system design to ensure consistency and reduce development errors.

Phase 4: Testing

This phase involves validating the functionality and accuracy of the developed data pipeline and analytics system. Data validation testing ensures that transformations are correctly applied and that the outputs from the Silver and Gold layers are accurate and consistent. Functional testing is conducted to verify that RA and MC risk calculations, financial provisioning, and inventory metrics are correctly generated. Integration testing ensures that data flows smoothly across different layers of the Medallion Architecture and between Azure services and Power BI dashboards. Any identified issues, such as incorrect calculations, missing data, or inconsistencies, are documented and resolved to improve the reliability and performance of the system.

Phase 5: Deployment

In this phase, the completed system is prepared for deployment within a simulated production environment on Microsoft Azure. The data pipeline is configured to enable automated data ingestion, transformation, and reporting processes. Power BI dashboards are published and made accessible to stakeholders for monitoring inventory risks in real time. This phase includes configuring access permissions, ensuring proper integration between Azure services, and validating that the entire workflow operates as intended. Although this project is implemented as a case study, the deployment phase demonstrates how the proposed solution can function in a real-world enterprise environment to support data-driven decision-making.

3.4 Project Planning Schedule

The project planning schedule gives an account of the tasks involved, their respective time frames, and the individual responsibilities that have been assigned to each team member during the development of the RA and IRM system. The use of a Gantt chart has been utilized in presenting the schedule of activities, which enables the understanding of how tasks are undertaken within each of the five phases of the project including the Introduction phase, Literature Review phase, Methodology phase, Analysis and Design phase, and System Implementation phase.

3.4.1 Gantt Chart

PPG PROJECT GANTT CHART

PROJECT TITLE	Recoverable Assets (RA) & Inventory Risk Management (IRM)	18-Mar-2026
PROJECT ADVISOR	Dr. Aryati	

S. No	TASK TITLE	ASSIGNED TO	START DATE	DUE DATE	DURATION (DAYS)	TASK STATUS
1	**Introduction**					
1.1	Project Overview	Lim Yu Han	18-Mar-2026	29-Mar-2026	1	Completed
1.2	Problem Background	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
1.3	Problem Statement	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
1.4	Project Objectives	Adib	18-Mar-2026	29-Mar-2026	11	Completed
1.5	Project Scope	Adib	18-Mar-2026	29-Mar-2026	11	Completed
1.6	Project Importance	Adib	18-Mar-2026	29-Mar-2026	11	Completed
2	**Literature Review**					
2.1	Introduction	Adib	18-Mar-2026	29-Mar-2026	11	Completed
2.2	Supply Chain Inventory Management	Ng Yu Hin	18-Mar-2026	29-Mar-2026	11	Completed
2.2.1	Supply Chain Management in Inventory	Ng Yu Hin	18-Mar-2026	29-Mar-2026	11	Completed
2.2.2	Inventory Management Logic	Ng Yu Hin	18-Mar-2026	29-Mar-2026	11	Completed
2.2.3	Financial Provisioning	Ng Yu Hin	18-Mar-2026	29-Mar-2026	11	Completed
2.3	Data Architecture	Auni	18-Mar-2026	29-Mar-2026	11	Completed
2.4	Data Modeling	Neo Li Xin	18-Mar-2026	29-Mar-2026	11	Completed
2.5	Enterprise Resource Planning	Adib	18-Mar-2026	29-Mar-2026	11	Completed
2.6	Technology Comparison	Ling Yu Qian	18-Mar-2026	29-Mar-2026	11	Completed
2.6.1	Cloud vs. On-Premise	Ling Yu Qian	18-Mar-2026	29-Mar-2026	11	Completed
2.6.2	ETL vs. ELT	Ling Yu Qian	18-Mar-2026	29-Mar-2026	11	Completed
2.6.3	Visualization Tools	Ling Yu Qian	18-Mar-2026	29-Mar-2026	11	Completed
2.7	Related Work and Case Studies	Auni	18-Mar-2026	29-Mar-2026	11	Completed
2.8	Comparison with Previous Studies	Auni	18-Mar-2026	29-Mar-2026	11	Completed
2.9	Microsoft Azure	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
2.9.1	Map of Azure Services to PPG Pipeline Layers	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
2.9.2	Azure Data Factory to Orchestrate Pipelines	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
2.9.3	Power BI Dashboard Delivery Integration	Lim Yu Han	18-Mar-2026	29-Mar-2026	11	Completed
2.10	Microsoft Power BI	Neo Li Xin	18-Mar-2026	29-Mar-2026	11	Completed
2.10.1	Data Connectivity and Integration	Neo Li Xin	18-Mar-2026	29-Mar-2026	11	Completed
2.10.2	Data Analysis Expressions (DAX)	Neo Li Xin	18-Mar-2026	29-Mar-2026	11	Completed
2.10.3	Interactive Visualization and Reporting	Neo Li Xin	18-Mar-2026	29-Mar-2026	11	Completed
2.11	Summary	Adib	18-Mar-2026	29-Mar-2026	11	Completed
3	**Methodology**					
3.1	Introduction	Lim Yu Han	30-Mar-2026	19-Apr-2026	19	In Progress
3.2	The Chosen Methodology	Auni	30-Mar-2026	19-Apr-2026	19	In Progress
3.3	Phases of the Chosen Methodology	Ng Yu Hin	30-Mar-2026	19-Apr-2026	19	In Progress
3.4	Project Planning Schedule	Neo Li Xin	30-Mar-2026	19-Apr-2026	19	In Progress
3.5	Summary	Ling Yu Qian	30-Mar-2026	19-Apr-2026	19	In Progress

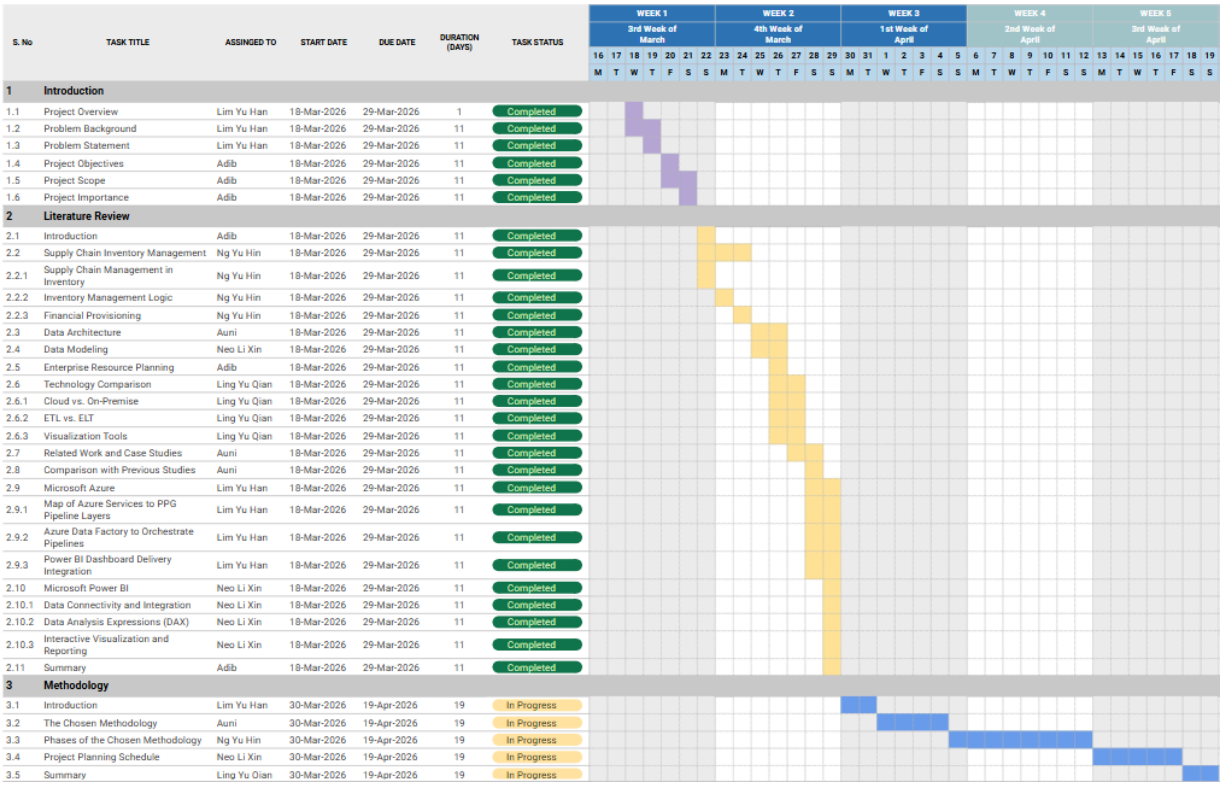


Figure 3.2 Project Gantt Chart Phase 1 to Phase 3

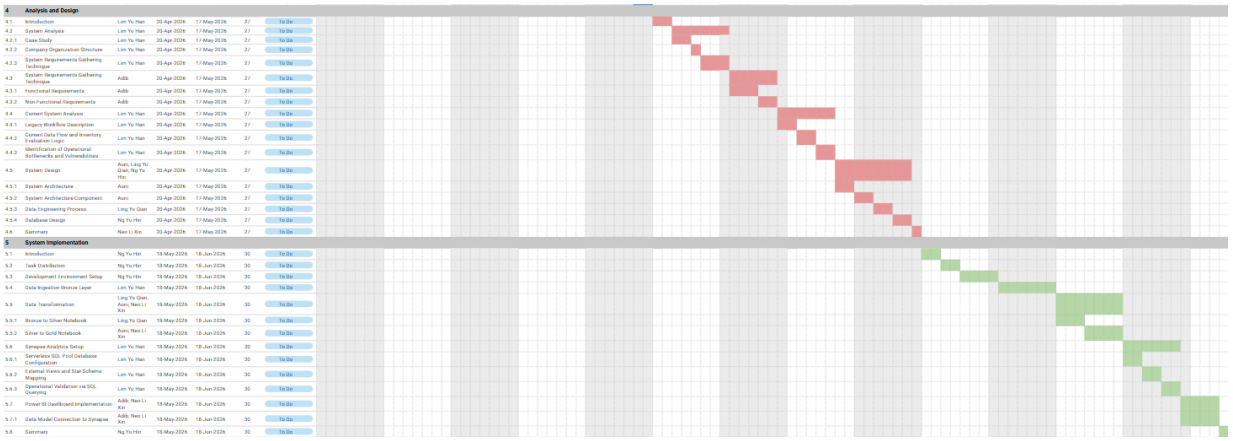


Figure 3.3 Project Gantt Chart Phase 4 to Phase 5

The following is an illustration of the project planning schedule for the development of RA/IRM system using Figures 3.2 and 3.3. The plan has been developed in the format of a Gantt Chart and involves the listing of project activities, the allocation of each task to team members, time periods, and completion stages during the fourteen weeks duration of project development process. The project includes five phases: Introduction, Literature Review, Methodology, Analysis and Design, and System Implementation.

Figure 3.2 shows Phases 1, 2, and 3 of project development that include Introduction, Literature Review, and Methodology respectively. Phases 1 and 2 are concerned with building the project foundation and carrying out a literature review. Phase 3 involves the identification of the research methodology and planning of project activities. The chart indicates that all the activities related to Phases 1 and 2 had been carried out successfully within the first two weeks of the project development. Activities relating to Phase 3 were planned to be accomplished up until the third week of April.

Phase 4 and Phase 5, as seen in Figure 3.3, are the Analysis & Design and System Implementation stages respectively. Requirements analysis, system architecture & DB design, data engineering operations, Azure data pipeline implementation, Synapse Analytics setup, and the creation of Power BI dashboards fall under these two phases. Their duration is from late April to mid-June, giving enough time for systems' development, testing, validation, and completing the project in general. A Gantt chart will be used as a tool for managing this project.

3.5 Summary

This chapter has described the methodological approach that was taken to design the PPG inventory risk management system based on the SDLC framework. In particular, to guide the project, the linear Waterfall model was selected. The main outcome of the choice of this particular methodology is a highly organized development roadmap that is important since the parameters of the project, such as the six raw operational datasets, or the desired outputs of RA and MC risks, were clearly defined in the first place.The methodology is a systematic sequence of six controlled steps which include Requirement Analysis, System Design, Implementation, Testing, Deployment, and Maintenance. Through setting clear deliverables at every level, the product of one phase is the input to the next. Indicatively, the existing constraints of disjointed data silos were analyzed to directly shape the next phase of creating a three-tier Medallion Architecture on Microsoft Azure.The strategic benefit and subsequent success of this Waterfall method are that it is phase-gated. Since the data pipeline architecture and ELT transformation logic need to be fully defined prior to any implementation, the methodology proactively mitigates the risk of design inconsistencies. Moreover, the single Testing step will make sure that the intricate logic of the financial provisioning of the RA and MC is strictly tested before the ultimate Power BI dashboards are launched. Finally, such a rigorous methodological approach will guarantee that the analytics system provided are not only reliable and scalable but also appropriately address the fundamental goal of moving the organization to a proactive and data-driven inventory approach.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

In Chapter 4, the comprehensive analysis and system design blueprints for the proposed automated Recoverable Assets (RA) and Inventory Risk Management (IRM) platform at PPG Industries Malaysia were presented. This chapter brings the high-level project lifecycle phases described in the project methodology down to the granular level, detailing the logic, functional boundaries, and architecture that is needed to build an end-to-end cloud data pipeline with Microsoft Azure and Power BI. The main goal is to get rid of the plant's current manual auditing processes and to implement an automatic system that will process non-uniform operational data to dynamically identify inventory obsolescence and measure the financial provision liabilities.

4.2 System Analysis

This section outlines the system analysis done on the proposed inventory risk management solution. The analysis starts with the description of the case study and system requirements elicitation techniques. The results of this analysis will be used to establish the functional and non-functional requirements for the proposed system.

4.2.1 Case Study

PPG Industries Malaysia is equipped with a chemical plant producing high tech industrial coatings, car finishes and building coatings. In this dynamic working environment, the management of raw material cycles is of great importance for the liquidity of the supply chain, the savings of overheads and the right balance sheet valuation. A primary operational

risk at the facility is inventory obsolescence, which is the continuous decrease in the number of products that are physically in the warehouse compared to the number that are being produced.

PPG has two different internal financial and operational risk categorizations to measure and control significant risks of dead, expiring or excess inventory.

- **Recoverable Assets (RA):** Raw materials that have a higher number of physical units on hand (based on the inventory_snapshot.csv) than the expected 12-month moving manufacturing consumption (based on the production_orders.csv). These materials are bound up in working capital, and should be considered quarterly, formally, and reviewed for means of recovery or reallocation to other production.
- **Magna Carta (MC):** There are sub-classes of RA, the most severe of which is material obsolescence. The category is broken down into two sub categories: MC Dead Stock (materials that have not had any manufacturing consumption for 9 months or more) and MC Expired (materials with the certified lot expiry date passed). Magna Carta inventory is a total asset loss in accordance with strict corporate compliance rules and requires a strict 100% financial write down provision today.

The data context of this problem space consists of six separate source files: materials.csv (structural material dimensions, safety margins, base costs), suppliers.csv (vendor metadata and reliability ratings), inventory_snapshot.csv (monthly localized warehouse stock balances across locations WH-A, WH-B, WH-C, WH-D, and WH-E), purchase_orders.csv (inbound supply chain and lead-time data), production_orders.csv (historical manufacturing material consumption), sales_orders.csv (outbound customer commitments). To address financial volatility and operational delays, these individual transactional components need to be connected in an automated cloud analytics system, which is achieved through system analysis.

4.2.2 System Requirements Gathering Techniques

Instead of using traditional user interviews, this project's requirements were gathered using an analytical and organized method designed for data engineering pipelines and case study frameworks. The techniques are:

- **Document Analysis and Business Logic Synthesis**: Analysis of enterprise inventory tracking documents was carried out to identify the operational boundaries of inventory

risks. To be more specific, risks of Recoverable Assets (RA) and Magna Carta (MC) were identified and taken out. Then, it will be converted into strict technical data transformation rules.

- **Dataset Schema Inspection**: Throughout all six core CSV files that were given, this team performed investigation and review on each file. It is to find the details for data ingestion and relatable requirements. So, this allows the team to identify such as primary keys, foreign keys etc. that are required to design the data pipeline.
- **Domain Benchmarking**: This team also gathers technical requirements for cloud architecture. It is by analyzing industry-standard frameworks and documented case studies of global manufacturing enterprises such as Epiroc and WNS. From there, it give the team more vision on technical specifications required to model the three-tier Medallion Architecture. Not to forget, this team can select the appropriate cloud stack.

4.3 System Requirements

This section outlines the technical and operational criteria that the proposed data analytics platform must satisfy. To provide a clear and structured framework for development and evaluation, these specifications are divided into two main categories:

- **Functional Requirements:** The team defines the specific capabilities, data processing workflows, and business logic that the system must actively execute.
- **Non-Functional Requirements:** The team specifies the architectural standards, performance benchmarks, scalability, and fault-tolerance mechanisms that govern the system's operational quality.

4.3.1 Functional Requirements

- The system must automatically ingest the six core CSV datasets from their source paths into the Bronze layer of Azure Data Lake Storage using Azure Data Factory orchestration.
- The system must utilize Azure Databricks to clean raw data.

- The system needs to calculate a rolling 12-month consumption forecast for each raw material to evaluate stock levels.
- The system needs to flag any raw material stock quantities exceeding the 12-month as a Recoverable Asset.
- The system must identify materials with zero operational consumption for 9 consecutive months or those past their expiration dates.
- The system must automatically map material shortages to specific customer sales orders.
- The system must deliver an interactive Power BI dashboard.

4.3.2 Non-Functional Requirements

- The system should automatically adjust its computing power and storage size based on how much ERP data is being processed, without needing people to manually upgrade servers.
- The system should organize data in a way that makes reports and dashboards run very fast, especially in Microsoft Power BI.
- The system must keep an original copy of all source data exactly as it was received.
- The system should continue from the failed step instead of restarting everything from the beginning if part of the pipeline fails.
- The Microsoft Power BI dashboard should feel smooth and responsive when users interact with it.

4.4 Current System Analysis

The current process for inventory management in PPG Industries Malaysia is analysed in this section. The analysis highlights the workflow employed for identifying RA and MC inventory risks, data flow between operational systems and problems with manual inventory evaluation. As part of this, it is important to understand the constraints of the existing process to uncover opportunities for improvement and requirements for the proposed data engineering solution.

4.4.1 Legacy Workflow Description

The inventory risk assessment and asset write down provision system at PPG Industries Malaysia (PIM) is currently manual, disjointed and spreadsheet-based, and is the current legacy operational system. This is done on a non-real time basis at the end of each fiscal quarter. The first step in the workflow is to manually download the isolated localized databases' separate transactional data logs and save the individual logs as localized desktop spreadsheets. The operators then have to mess with the disjoined files to merge them together using local spreadsheet software, straining local computing resources. A complicated nested array formula would have to be written to cross-reference the actual stock level with the manufacturing history and determine the 12-month moving average consumption of the raw material for each row. This flow needs to be adhered to line by line across thousands of material batches, resulting in a lot of operational friction and labour time that could be better spent on optimizing the supply chain.

4.4.2 Current Data Flow and Inventory Evaluation Logic

The manual system tries to assess risks to inventory based on information from various disjointed files, but there is no integrated database structure or formal entity relationships. Stock quantities are read from the inventory snapshot file, and the unit cost file should be read from the master materials sheet. Recoverable Assets are identified by manually calculating total consumption from historical production orders in the previous 12-months and then marking the material as an excess asset if there is more on hand than this computed amount. To determine Magna Carta Dead Stock the operator must perform a row-by-row visual inspection of production logs to find material identifiers that have zero usage flags for nine months or more. Finally, Magna Carta Expired stock is determined by performing a static inventory check for whether or not a batch has passed its certified lot expiry date. After these groups are manually separated, the financial provision is then calculated on another sheet, using obsolete quantities multiplied by the costs of the base unit of the file, which is very dependent on the structural integrity of the unmanaged files.

4.4.3 Identification of Operational Bottlenecks and Vulnerabilities

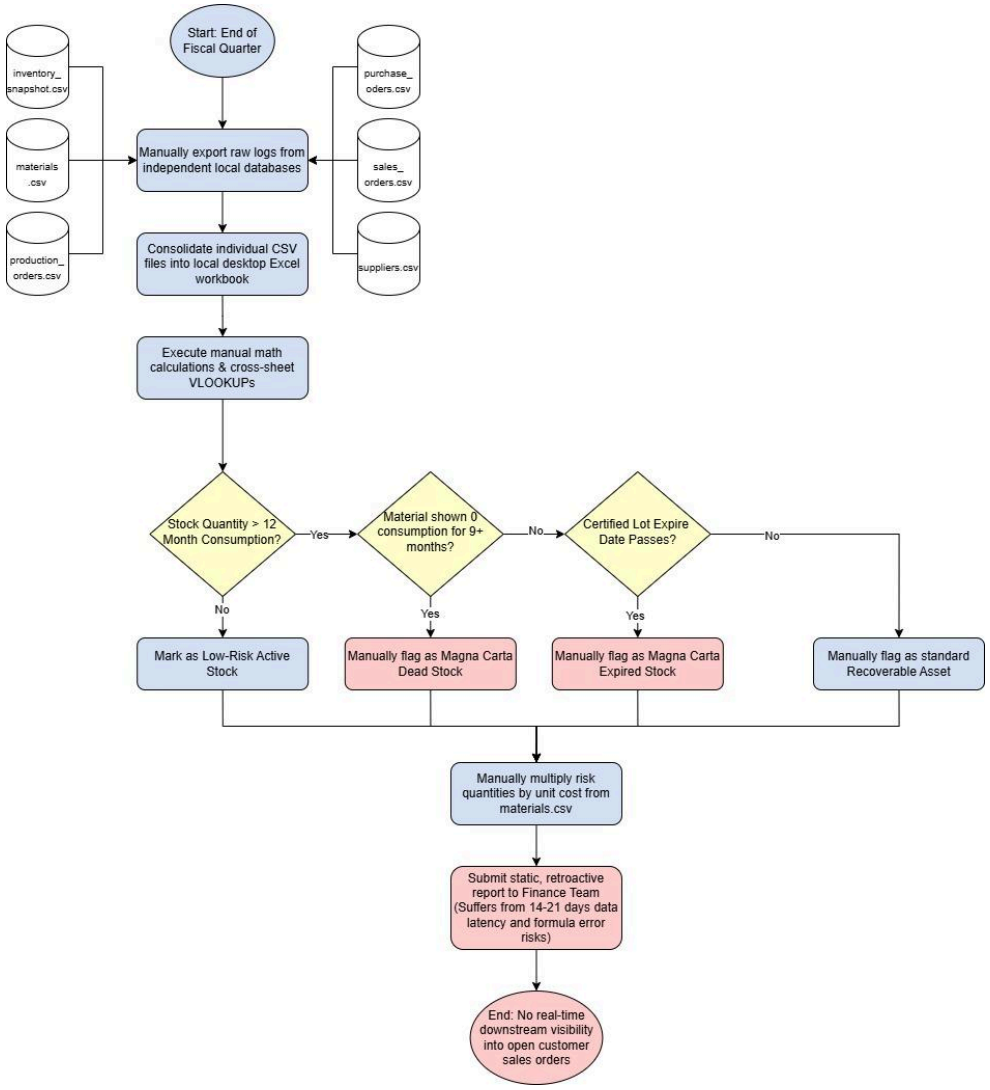


Figure 4.2: Flowchart of the legacy manual inventory evaluation process highlighting conditional risk verification checkpoints, severe data latency gaps, and human calculation vulnerabilities.

The current legacy inventory evaluation process PPG uses is shown in Figure 4.2. The process involves staff manually gathering data from a number of unconnected data sources, manually calculating inventory risk values in spreadsheets, and making manual checks of RA and MC conditions. This process adds multiple points of operational delays and risk to the supply chain, and to the financial reporting process.

An in-depth examination of this manual architecture uncovers significant systemic weaknesses that impact supply chain performance and financial reporting precision. The first is the data latency and data siloing from the architecture because all the data streams are derived from procurement information, tracking information from the warehouse and manufacturing logging systems, which are independent and unconnected. These logs can be merged periodically, every few weeks, and they are static and retroactive, meaning that the warehouse team will never know which lots are expiring or when material they need is going to be out of stock and disrupt their ongoing production schedule. Furthermore, spreadsheets are prone to errors due to human data entry and manual calculations, and small errors in the calculations, such as in the nested formulas, external workbook links or accidental deletion of rows, could affect the final calculations and potentially the integrity of financial provisions reporting to corporate stakeholders. Lastly, the legacy system doesn't provide a trace of downstream effects, and the warehouse team cannot relate a shortage in raw materials to a customer commitment that is on the go. If a material is below the safety level then nothing is in place to cross reference lead times with open sales orders and to predict which sales orders may be at risk of being delayed, which can have a negative impact on client relationships and plant reliability.

4.5 System Design

This section illustrated the conceptual and physical data architecture, its data engineering processes and database design for this project.

4.5.1 Conceptual Data Architecture

The conceptual architecture shown in Figure 4.3 is organised into three layers which are Data Source, ETL Process, and Data Visualization. The Data Source layer consists of six CSV files representing the organisation's core operational data. The files are extracted into the ETL Process layer, which follows the Medallion Architecture pattern, and stored as-is in the Bronze layer of the Data Lake. The data is then transformed and moved into the Silver layer, where it is cleaned and validated. Then, undergoing a further transformation into the Gold layer, where it is refined into business-level data. This Gold layer data is subsequently loaded into the PPG Data Warehouse, which consolidates the data for downstream use. From the Data Warehouse, the Data Visualization layer uses Power BI as the reporting and analysis tool to retrieve and visualise the data to produce the PPG Analytics Dashboard for end users.

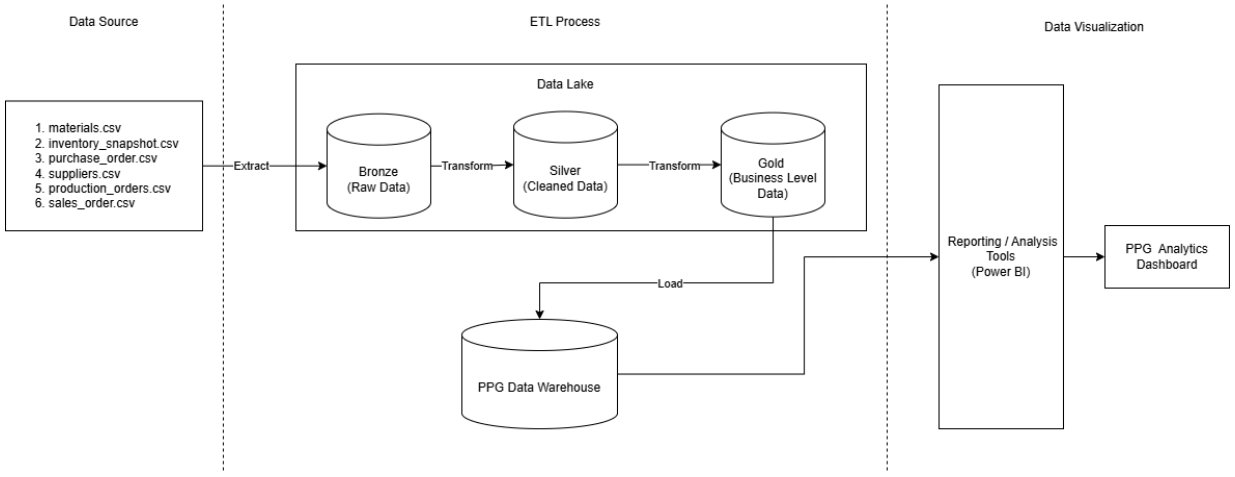


Figure 4.3: Conceptual Data Architecture

4.5.2 Physical Data Architecture

Figure 4.4 illustrates the cloud-based system architecture of the PPG data engineering pipeline, built on Microsoft Azure and following the Medallion Architecture pattern. The pipeline begins with six CSV files extracted via Azure Data Factory, which orchestrates the data pipeline. Within Azure Databricks, the data is refined through a Data Lake organised into Bronze, Silver, and Gold layers, using two notebooks. First is the Bronze to Silver notebook which cleans and validates raw data. Second is the Silver to Gold notebook which applies business logic to produce analysis-ready datasets.

Once refined, the Gold layer data is loaded into Azure Synapse Analytics, which acts as the central data warehouse. Then, Synapse is connected to Power BI to deliver interactive dashboards covering seven key reporting areas, including materials below reorder level, recoverable assets, dead stock, and stockout risk for open sales orders, among others. Throughout the architecture, Azure Key Vault is implemented to securely manage credentials and access keys, ensuring that only authorised services can interact with the pipeline.

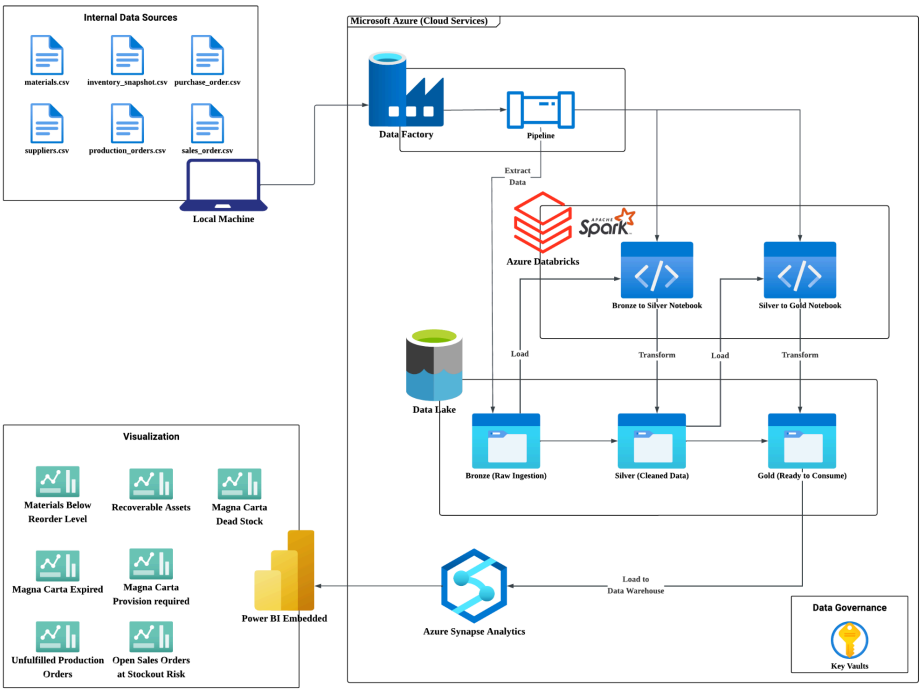


Figure 4.4: Physical Data Architecture

4.5.3 Data Engineering Process

This data engineering process for the Recoverable Assets (RA) and Inventory Risk Management (IRM) system is designed using a modern Extract, Load, Transform (ELT) approach that leverages a three-tier Medallion Architecture on Microsoft Azure. Data Ingestion Strategy

The data pipeline starts with six synthetic datasets from different sources as its targets, which include materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv—the heart of the PPG supply chain. Controlling the flow of this data, Azure Data Factory is used as the main orchestration tool, with the ability to synchronize the ingestion and transformation processes. Bronze Layer (Ingestion & Cleaning Rules)

The Azure Data Factory copies the raw CSV files from the source to the Bronze layer (Azure Data Lake Storage Gen2 in this case). Auditable Raw Storage: Data is stored as is, without hard rules being applied to the data as it is entered. Pipeline Resilience: When you intentionally don't transform the data, you're able to keep a historic record of the data, and if the business logic later needs to be reconfigured, the entire pipeline can be reprocessed from this source of raw data. This is a silver layer (Transformation Business Logic).

After the data is staged in Azure Databricks, the complex business logic is applied to the data and joins it together and cleans it to create the silver layer. To enforce PPG's strict inventory logic, Databricks efficiently performs window operations and date comparisons on the very large sets of data. The transformation rules applied are: The following transformation rules will be applied:

Rolling Consumption Calculation: The system will calculate the actual 12-month rolling manufacturing consumption for each material based on historical data provided in the production_orders.csv file.

Recoverable Assets (RA) Identification: It is the pipeline that compares the forecast of consumption for the last 12 months with the physical units that are on hand, and these physical units are taken from inventory_snapshot.csv. If the inventory level exceeds the forecast level for the next year, it is classified and marked as an RA (Excess Stock).

Magna Carta (MC) Risk Detection: The system performs a more extensive obsolescence classification, which determines a serious loss of assets, and is broken down into two sub-classifications:

MC Dead Stock: Materials with no manufacturing use whatsoever for 9 months or more.

MC Expired: Materials that have a certified lot expiration date that has passed.

Auto-Financial Provisioning: If the material is flagged as MC, then the standard accounting principle is that it is no longer an asset. The Silver layer logic dynamically computes a 100% financial write-down, which is the number of obsolete physical quantities multiplied by the base cost of the material in materials.csv.

4.5.4 Database Design

To effectively manage and analyse inventory operations at PPG, the database is designed using an Entity Relationship Diagram (ERD) approach. This relational database model organizes operational data into structured entities and establishes relationships between them to ensure consistency, reduce redundancy and support efficient querying for inventory analysis as well as risk monitoring.

The ERD consists of eight main entities which are Materials, Inventory, Warehouse, Suppliers, PurchaseOrder, PurchaseOrderItem, SalesOrder, SalesOrderItem, and Customer. These entities represent the core business processes involved in inbound procurement, warehouse stock management, and outbound sales fulfillment.

This design enables the system to track material movement from suppliers into warehouses and from warehouses to customers, while maintaining stock visibility and supporting inventory risk detection such as excess stock, dead stock, and expired stock.

Table Name	Primary Key (PK)	Foreign Keys (FK)	Core Attributes
Materials	materialId	–	materialName, category, baseUnitCost, shelfLifeDays, safetyStockLevel
Inventory	inventoryId	materialId, warehouseId	lotExpiryDate, physicalStockQuantity, rollingConsumption, financialProvision
Warehouse	warehouse_id	–	location_name
Suppliers	supplier_id	–	supplier_name, vendor_rating, region
PurchaseOrder	poId	supplier_id	orderDate, status

PurchaseOrderItem	poItemId	poId, materialId	quantityOrdered, quantityReceived, leadTimeDays
SalesOrder	salesOrderId	customer_id	orderDate, fulfillmentStatus
SalesOrderItem	salesOrderItemId	salesOrderId, materialId	orderQuantity
Customer	customer_id	–	customer_name, industry_type

Table 4.1: Database Structure with Entities and Attributes

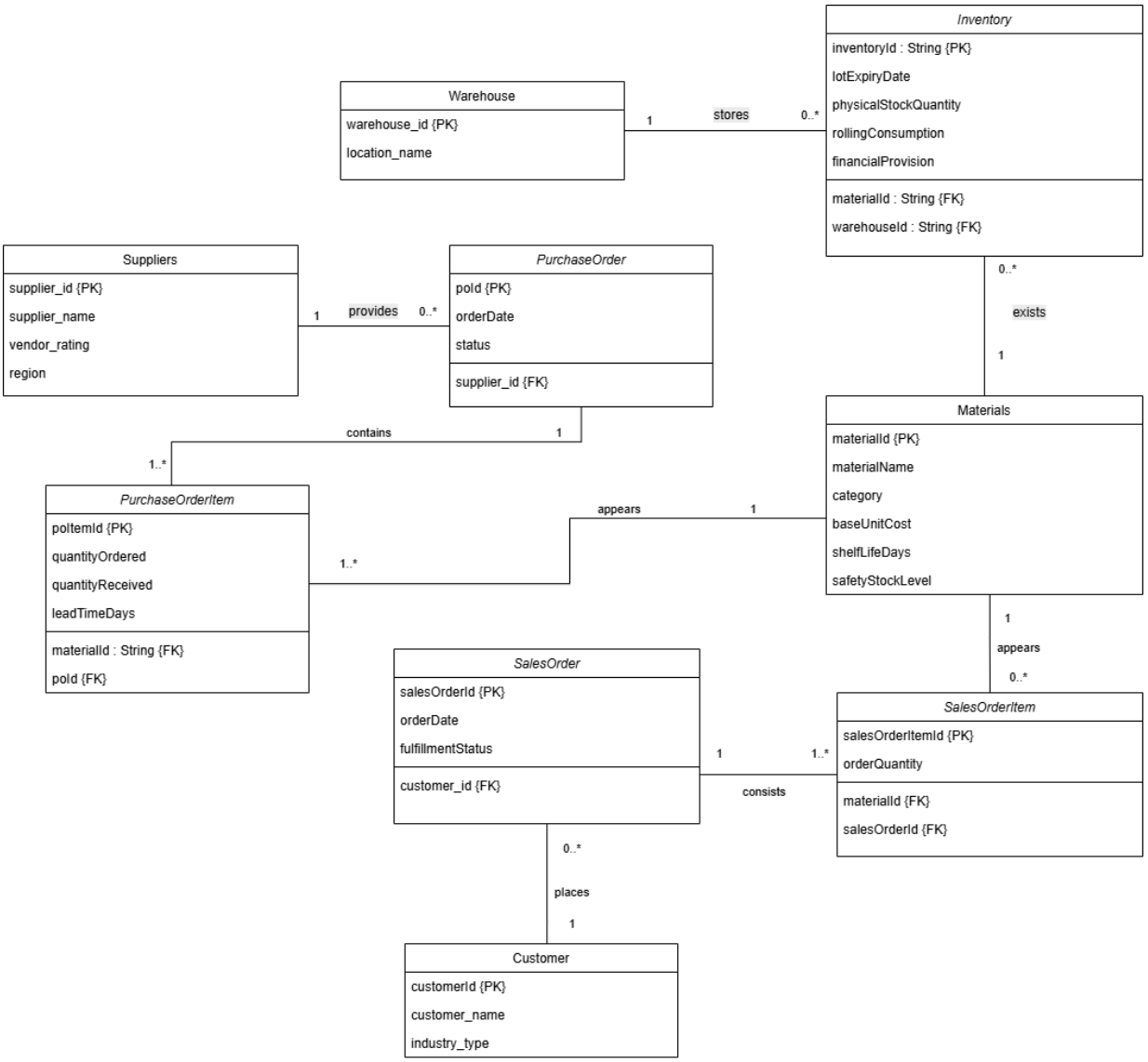


Figure 4.5: Entity Relationship Diagram (ERD) of Inventory Management System

Table Relationships and Multiplicities

The database enforces relational integrity using **One-to-Many (1:N)** relationships across all connected entities.

1:N Warehouse to Inventory

A single warehouse can store multiple inventory records, but each inventory record belongs to only one warehouse.

Relationship:

Warehouse (1) → Inventory (0..*)

1:N Materials to Inventory

A single material can exist in multiple inventory records across different warehouses or batches.

Relationship:

Materials (1) → Inventory (0..*)

1:N Suppliers to Purchase Orders

One supplier can provide multiple purchase orders, while each purchase order belongs to only one supplier.

Relationship:

Suppliers (1) → PurchaseOrder (0..*)

1:N Purchase Order to Purchase Order Item

A purchase order can contain multiple material items.

Relationship:

PurchaseOrder (1) → PurchaseOrderItem (1..*)

1:N Materials to Purchase Order Item

A material can appear in many purchase order items.

Relationship:

Materials (1) → PurchaseOrderItem (1..*)

1:N Customer to Sales Orders

A customer can place multiple sales orders.

Relationship:

Customer (1) → SalesOrder (0..*)

1:N Sales Order to Sales Order Item

A sales order can consist of multiple ordered materials.

Relationship:

SalesOrder (1) → SalesOrderItem (1..*)

1:N Materials to Sales Order Item

A material can appear in multiple customer orders.

Relationship:

Materials (1) → SalesOrderItem (0..*)

Data Granularity

The database maintains data at the transactional level to preserve detailed operational records.

For inventory, the grain is defined as:

One row per material, per warehouse, per lot/batch.

This level of granularity is important because the same material may have multiple batches with different expiry dates and stock quantities.

For purchase and sales transactions, the grain is:

- One row per purchase order item
- One row per sales order item

This ensures accurate tracking of inbound and outbound material flow, enabling the system to calculate stock consumption, monitor lead times, and identify potential inventory risks.

4.6 Summary

This chapter discussed the analysis and design of the RA and IRM system for PPG Industries Malaysia. In the analysis stage, it was considered necessary to look into the inventory management environment, organizational structures, system requirements, and shortcomings of the current manual method of inventory management. Several crucial problems have been found in the analysis stage; for example, information silos, delay in reporting, manual calculation, high probability of errors committed by the human operator, inability to detect obsolete inventory, and stock-out problems.

A cloud-based system architecture was developed based on the requirements analysis using Microsoft Azure technologies and the Medallion Architecture approach. The solution utilizes Azure Data Factory to manage the data flow, Azure Data Lake Storage to store the data in a multilayered manner, Azure Databricks to transform the data and apply the business logic, Azure Synapse Analytics as a serving layer and Power BI for interactive data visualization. The data engineering was implemented with the usage of the ELT method allowing for automatic identification of RA, MC Dead Stock, MC Expired items and financial provisioning computation.

Moreover, the Fact Constellation Schema was introduced in order to facilitate effective data analysis and reporting. The dashboard design allows the company stakeholders to get access to the current status of the inventory health, potential stockouts, financial liabilities and customer orders at risk.

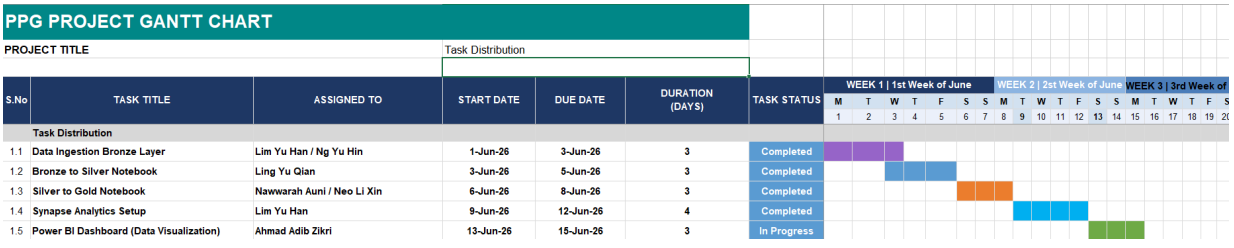
CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Introduction

This section outlines the actual implementation and system development of the end-to-end cloud analytics pipeline for the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) project. This phase captures the actual deployment of data engineering processes in the Microsoft Azure ecosystem, based on architectural blueprints and dimensional data models created in earlier phases. It includes the entire data lifecycle of the data moving from the automatic ingestion of raw operational data to the data lakehouse, passing through the strict transformation steps of the Medallion Architecture and finally to the high-performance visualization layer. The resulting infrastructure provides a secure, scalable and automated solution to the risks of inventory obsolescence.

5.2 Task Distribution



PPG PROJECT GANTT CHART

PROJECT TITLE: Task Distribution

S.No	TASK TITLE	ASSIGNED TO	START DATE	DUE DATE	DURATION (DAYS)	TASK STATUS
	Task Distribution					
1.1	Data Ingestion Bronze Layer	Lim Yu Han / Ng Yu Hin	1-Jun-26	3-Jun-26	3	Completed
1.2	Bronze to Silver Notebook	Ling Yu Qian	3-Jun-26	5-Jun-26	3	Completed
1.3	Silver to Gold Notebook	Nawwarah Auni / Neo Li Xin	6-Jun-26	8-Jun-26	3	Completed
1.4	Synapse Analytics Setup	Lim Yu Han	9-Jun-26	12-Jun-26	4	Completed
1.5	Power BI Dashboard (Data Visualization)	Ahmad Adib Zikri	13-Jun-26	15-Jun-26	3	In Progress

WEEK 1 | 1st Week of June — WEEK 2 | 2st Week of June — WEEK 3 | 3rd Week of

Figure 5.0 Gantt Chart for Task Distribution

To efficiently handle the automated pipeline,task distribution is conducted as shown in Figure 5.0.This encourages each assignee to keep track of time and progress at all times

Data Ingestion Bronze Layer

- **Assignee:** Lim Yu Han and Ng Yu Hin

- **Description:** Responsible for architecting the initial data extraction process. This involved configuring automated pipelines such as Azure Data Factory to securely ingest raw operational data files and land them in their immutable, original state into the data lake's Bronze layer.

Bronze to Silver Notebook

- **Assignee:** Ling Yu Qian
- **Description:** Tasked with the initial data cleansing and transformation phase using Databricks. This role required writing data processing notebooks to clean the raw data, handle missing values, enforce correct data types, remove duplicates, and load the validated datasets into the Silver layer.

Silver to Gold Notebook

- **Assignees:** Nawwarah Auni Binti Nazrudin and Neo Li Xin
- **Description:** Entrusted with implementing the core business logic. This involved writing advanced transformations to calculate metrics such as the 12-month rolling consumption, generate Recoverable Asset (RA) and Magna Carta (MC) risk flags, and structure the refined data into a Fact Constellation Schema before loading it into the Gold layer.

Synapse Analytics Setup

- **Assignee:** Lim Yu Han
- **Description:** Managed the data serving infrastructure. This task involved configuring Azure Synapse Analytics workspaces and serverless SQL pools to act as the optimized bridge between the structured Gold layer data and downstream business intelligence tools.

Power BI Dashboard (Data Visualization)

- **Assignee:** Ahmad Adib Zikri Bin A.Mazlam
- **Description:** Led the development of the front-end user experience. This involved connecting Microsoft Power BI to the serving layer to build an interactive,

high-performance dashboard that translates the underlying data models into actionable visual insights for executives.

5.3 Development Environment Setup

The implementation of the PPG Inventory Risk Management pipeline relies on a robust and scalable cloud infrastructure. The development environment was provisioned entirely within Microsoft Azure, utilizing a single, centralized Resource Group (my-resource-group) to manage the lifecycle, integration, and security of all related architectural components.

Azure Services Provisioned

To construct the end-to-end Medallion Architecture, the following core Azure resources were deployed:

- **Azure Data Lake Storage Gen2 (ppgprojectdl):** Serves as the foundational enterprise data lake. It is configured with a hierarchical namespace to efficiently store and manage the raw, cleansed, and curated datasets across the Bronze, Silver, and Gold layers.
- **Azure Data Factory (ppgprojectdf):** Acts as the primary data orchestration and ingestion service. It is responsible for establishing automated pipeline connections to extract the operational source files such as inventory snapshots and purchase orders and load them into the Bronze layer.
- **Azure Databricks (dbw-ppgproject):** Deployed as the core computational engine. This collaborative, Apache Spark-based environment hosts the Python/PySpark notebooks used for executing complex data cleansing, schema enforcement, and the calculation of inventory risk metrics for the Silver and Gold layers.
- **Azure Key Vault (ppgprojectkv):** Implemented to ensure strict security governance. It securely stores sensitive credentials, secrets, and connection strings required for Data Factory and Databricks to authenticate and interact seamlessly without hardcoding passwords.
- **Azure Synapse Analytics:** Functions as the high-performance serving layer. It provides serverless SQL pools to query the highly structured Gold layer Fact Constellation schema, acting as the optimized bridge to the Power BI dashboard.

Resource Group Setup and Configuration

The visual evidence of the provisioned development environment within the Azure Portal is documented below.

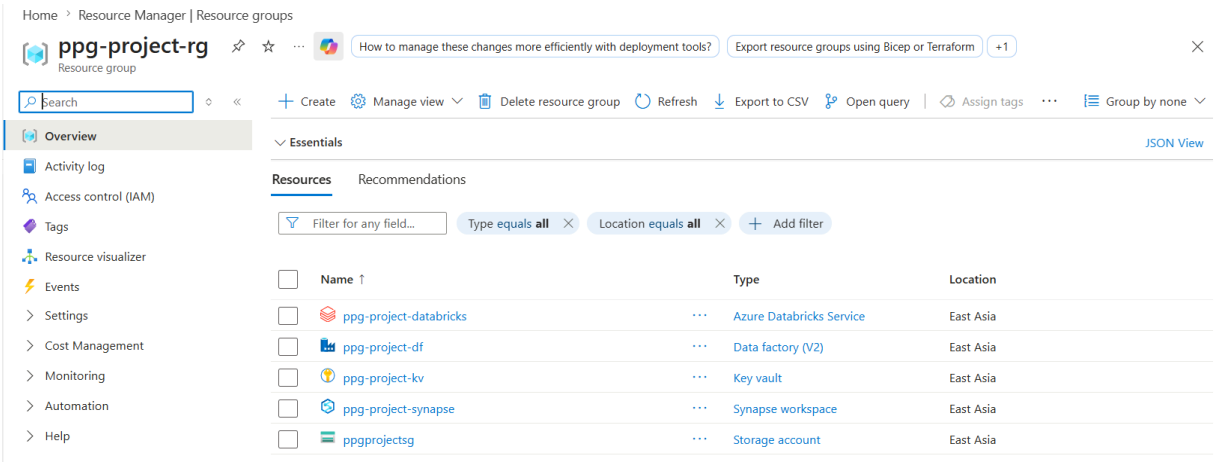


Figure 5.1: Overview of the primary Azure Resource Group, displaying the successfully deployed Data Lake, Data Factory, Key Vault, and Databricks service.

As illustrated in Figure 5.1, the primary Azure Resource Group successfully hosts all the interconnected components required for the pipeline, including the Data Lake, Data Factory, Key Vault, and Azure Databricks service.

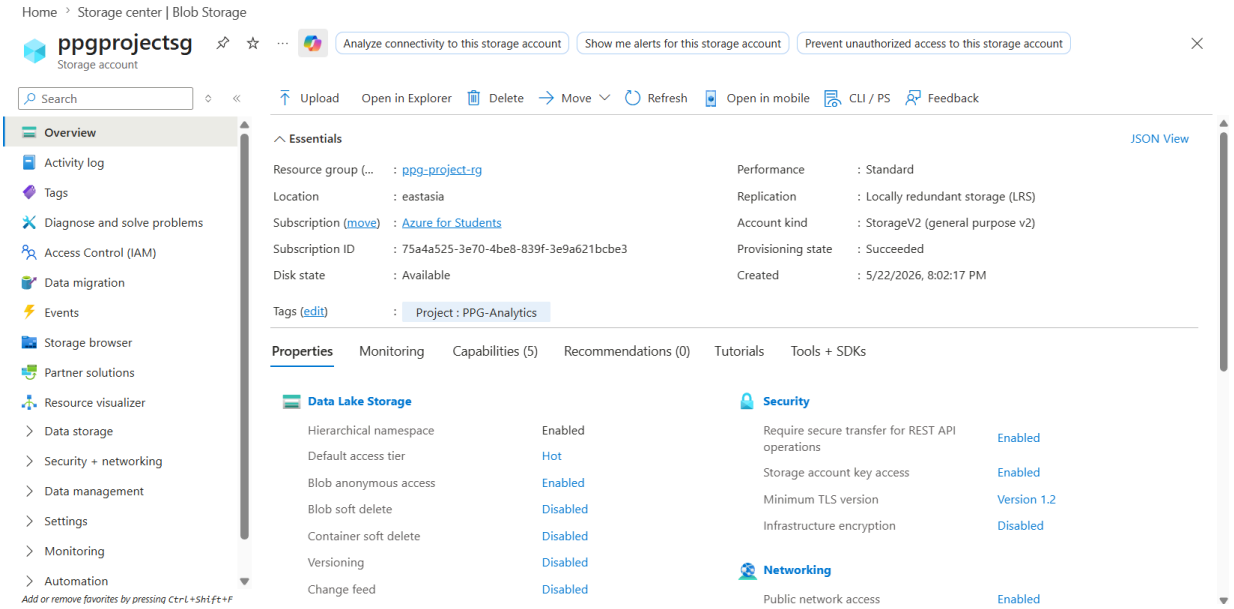


Figure 5.2: Azure Storage Account (Overview)

To handle the storage of the Medallion Architecture layers (Bronze, Silver, and Gold), a centralized storage account was provisioned. Figure 5.2 displays the ppgprojectsg storage account

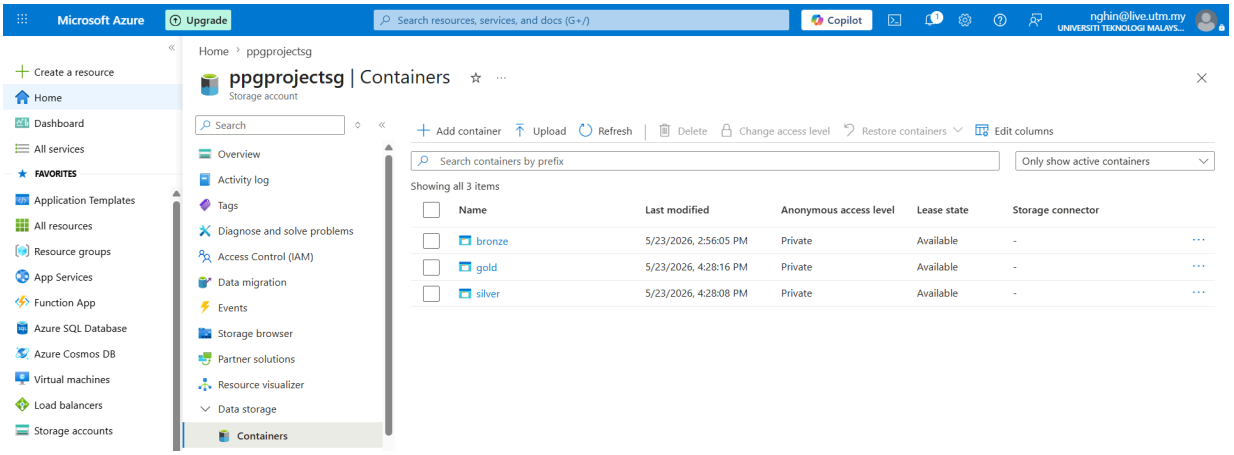


Figure 5.3: Azure Storage account (ppgprojectsg) with Data Lake Storage Gen2 capabilities enabled for scalable cloud storage.

As shown in Figure 5.3, Azure Data Lake Storage Gen2 capabilities enable scalable, hierarchical cloud storage for the raw and processed CSV datasets.

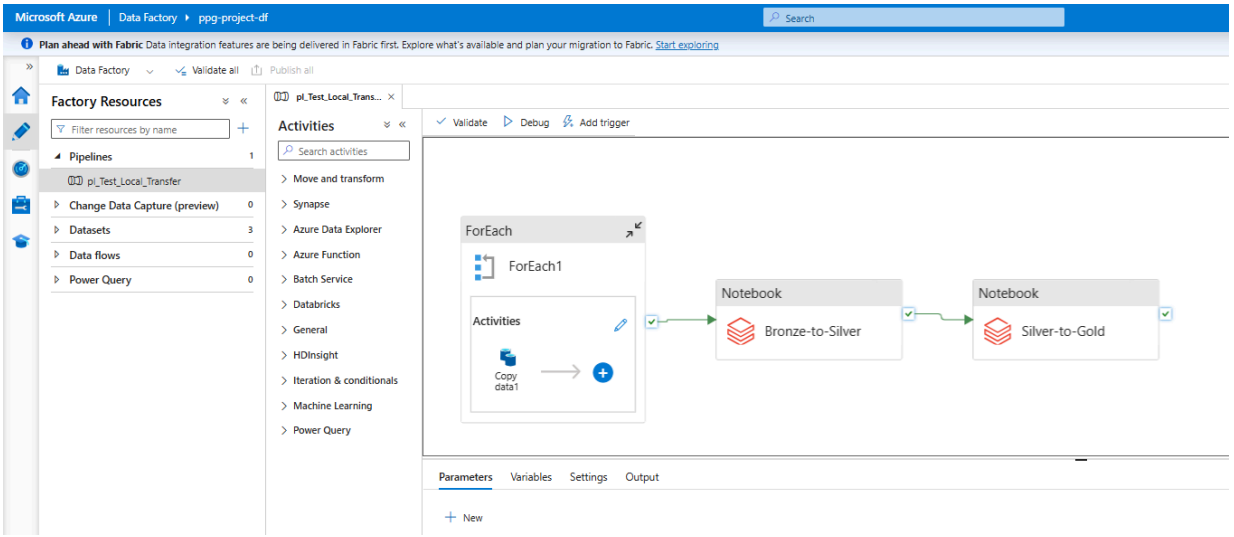


Figure 5.4: Azure Data Factory (ppgprojectdf) instance, provisioned for pipeline orchestration and data integration tasks.

For the orchestration of data movement, Figure 5.4 demonstrates the provisioned Azure Data Factory instance (ppgprojectdf). This service is configured to execute automated pipeline triggers and handle data integration tasks across the cloud environment.

.

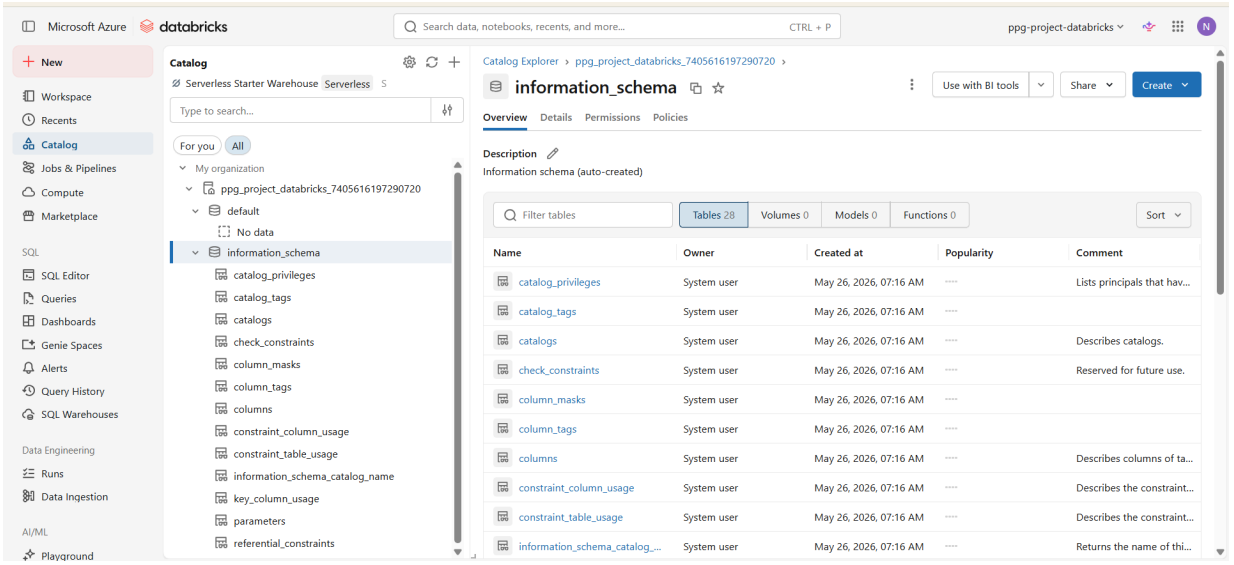


Figure 5.5: Azure Databricks workspace (dbw-ppgproject) for executing PySpark data transformations and inventory risk metric calculations.

Figure 5.5 illustrates the Azure Databricks workspace (dbw-ppgproject). This Apache Spark-based environment serves as the core computational engine, where the Python/PySpark

notebooks are hosted to execute data cleansing, schema enforcement, and the complex calculations for the RA and MC inventory risk metrics

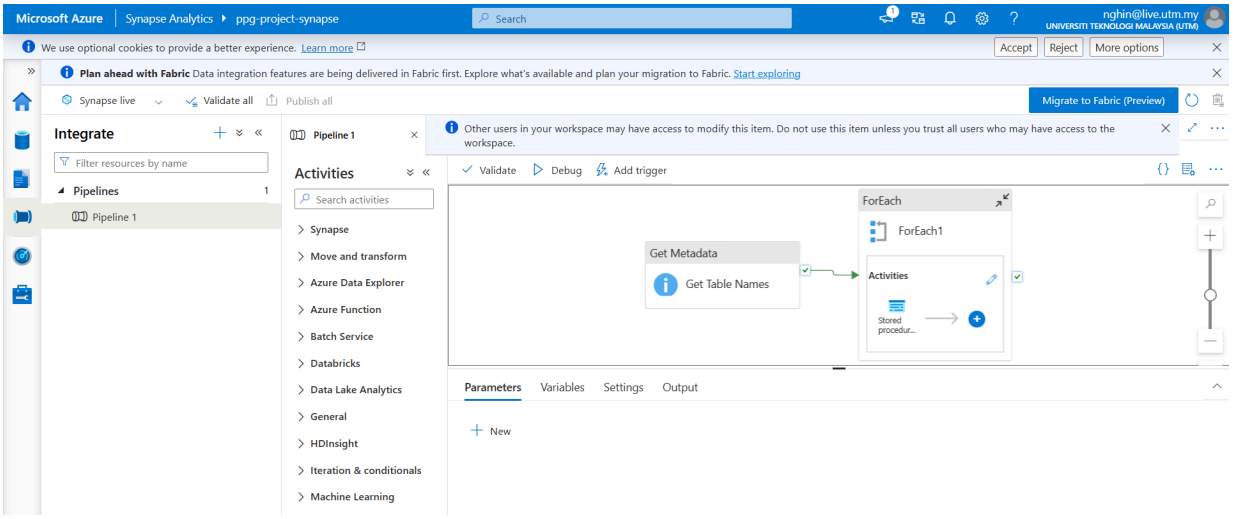


Figure 5.6: Azure Synapse Studio interface

As shown in Figure 5.6, Azure Synapse Analytics (ppg-project-synapse) combines data integration, enterprise data warehousing, and big data analytics into one platform. It acts as the high-performance "Serving Layer."

5.4 Data Ingestion Bronze Layer

The first part of the system implementation is on orchestrating data ingestion from the raw data repositories into the cloud environment. The migration process was automated by designing an Azure Data Factory (ADF) pipeline called ‘pl_Test_Local_Transfer’. The ingestion efficiency is optimized by combining a ForEach control activity with a dynamic Copy Data task. This structural option allows multiple source datasets of multiple data sources to be processed iteratively instead of sequentially, and therefore at a faster rate. The pipeline defines the file paths and connects to the source environment and safely pushes data into the cloud landing zone.

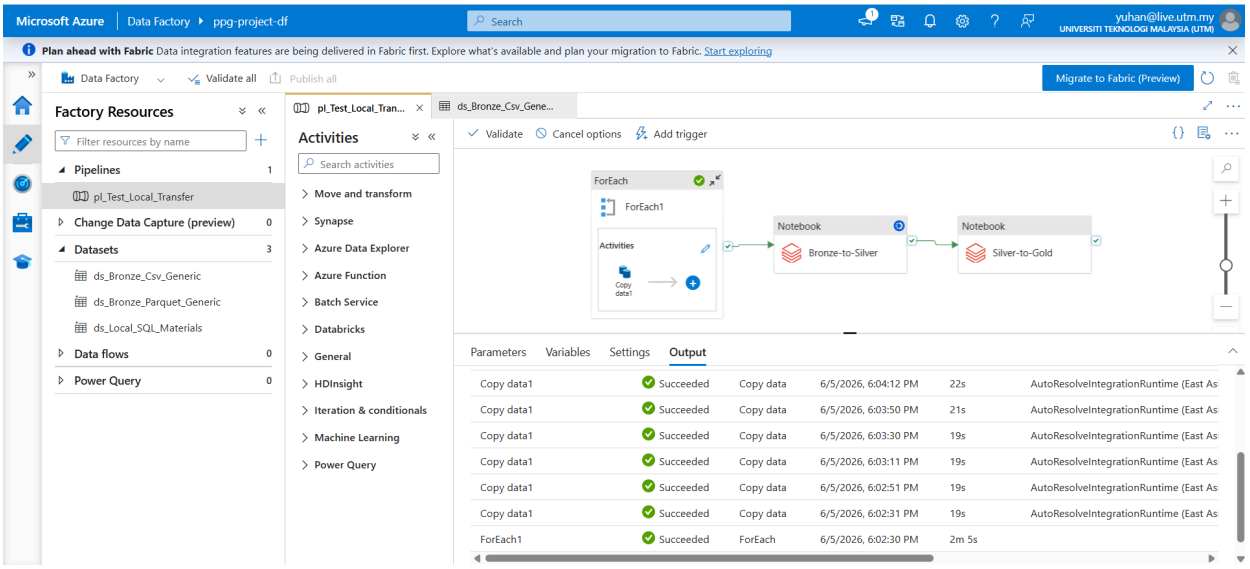


Figure 5.7: Azure Data Factory pipeline execution for automated data ingestion and Medallion Architecture processing.

The Azure Data Factory pipeline in Figure 5.7 orchestrates the entire end-to-end data engineering pipeline. The pipeline starts with a ForEach activity that processes each of several source data sets and runs a Copy Data operation to import raw CSV files into Azure Data Lake Storage's Bronze layer. Once the ingestion stage is successful, the pipeline automatically runs the Bronze-to-Silver notebook to cleanse and enforce the schema, then the Silver-to-Gold notebook to implement business logic and generate the dimensional model. The successful execution status shown in the output pane confirms that all ingestion activities completed without errors.

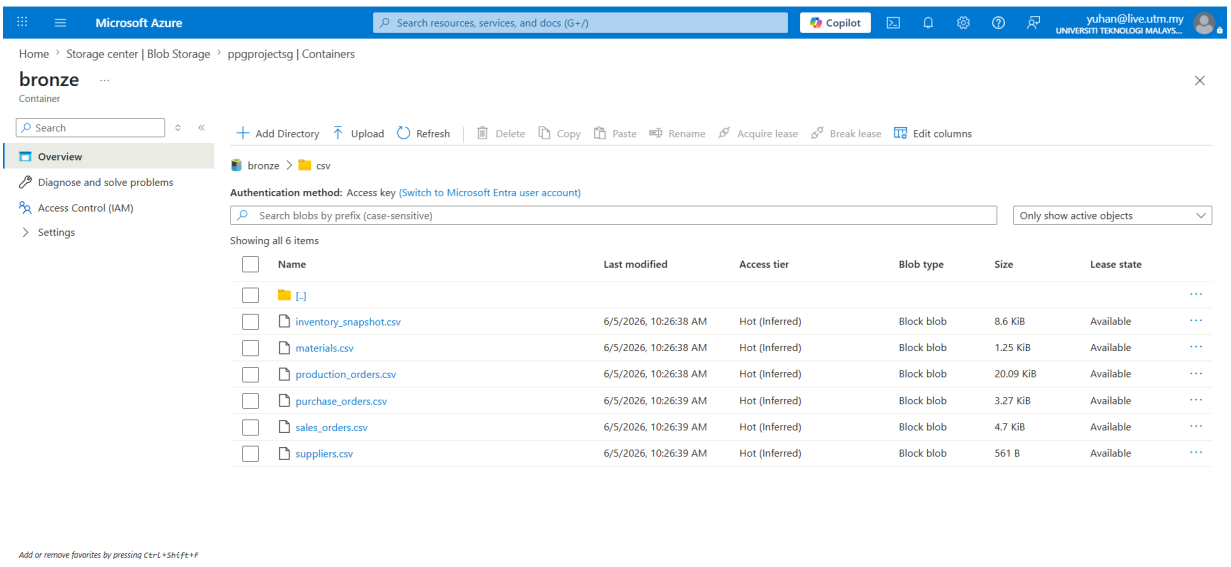


Figure 5.8: Raw operational datasets stored in the Bronze layer of Azure Data Lake Storage Gen2

In the Bronze layer, the results of successful completion of the ingestion pipeline can be seen, as shown in Figure 5.8. Inventory_snapshot.csv, materials.csv, production_orders.csv, purchase_orders.csv, sales_orders.csv and suppliers.csv are placed in the Azure Data Lake Storage Gen2 as CSV files. At this point, no transformations are performed on the raw data, maintaining its immutability and enabling it to be audited, reprocessed, and traced back to its origin or data lineage. This will follow the Medallion Architecture practice of preserving source data prior to later cleansing and transformation in the Silver layer.

5.5 Data Transformation

The Medallion Architecture refines raw, operational data through a rigorous two-step process in Azure Databricks to transform it into the data used for analytics. First, it systematically cleans six core datasets in the Bronze to Silver notebook and trims whitespaces, normalizes null values, removes duplicate records and enforces structural schemas by converting to the efficient Delta format. The Silver to Gold notebook then uses Apache Spark to execute complex business logic and produce important business metrics (KPIs) on two main analytical tables. The `fact_inventory_status` table combines material level data to derive consumption figures and alerts to severe inventory risks such as Recoverable Assets (RA), Magna Carta (MC) dead stock, and Magna Carta (MC) expired stock, and provides automatically calculated 100% financial write-down provisions for obsolete assets. Last but not least, the `fact_order_fulfillment_risk` table extends these learnings by correlating material shortages with production plans and pending orders, allowing for the detection of specific customer orders that could be in danger of being delayed.

5.5.1 Bronze to Silver Notebook

This phase deals with data cleansing and transformation that starts out in Azure Databricks. The notebook is tasked with systematically cleaning the raw operational data and putting the cleaned data into the Silver layer.

- Data Cleaning Rules applied to the data.

The notebook uses a self-made function called clean_staging_data which goes through every data column in the notebook.The notebook includes a custom function called clean_staging_data which loops through all data columns. If it is determined to be a string type, then a `trim()` function is used on each column to remove any trailing or leading whitespaces from the text, thus ensuring it is formatted the same way.

Automated Iteration: An execution loop reads the raw `parquet` files in the Bronze container for each of the six core tables (`materials`, `suppliers`, `inventory_snapshot`, `production_orders`, `purchase_orders`, and `sales_orders`), and performs the cleaning rules uniformly.

- The handling of null values and duplicates.

Null Value Normalization: The script is designed to detect if there is any empty string value entered into the cell, and it will set the value of that cell to `None` (Null) to prevent any down stream calculation issues in financial provisions.

Absolute duplicate rows that could have been created while importing the raw data from legacy systems are properly eliminated with a `dropDuplicates()` command.

- Schema Enforcement

Data from the Bronze layer is explicitly rewritten from the `parquet` format into the very efficient `delta` format in Delta Lake Transition.

Strict Data Typing: The notebook requires that data be of a certain type because of .option("overwriteSchema", "true") to resolve schema mismatch and/or damaged raw data instances. Upon overwriting Silver tables, this command will force the tables to safely replace the old or corrupted schemas with the newly validated schemas.

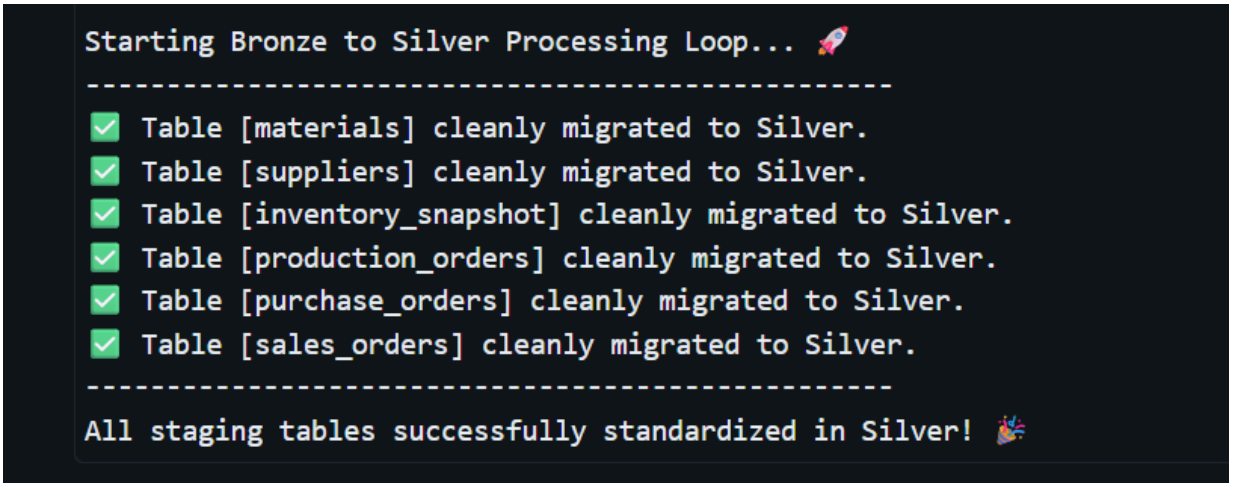


Figure 5.9: Bronze to Silver Processing Loop Execution Log

The Azure Databricks console execution log in Figure 5.9 shows that the Bronze to Silver data processing loop worked. As displayed in the output, the six core operational datasets (materials, suppliers, inventory_snapshot, production_orders, purchase_orders, and sales_orders) were processed one at a time without any systemic failures happening along the way. This console output provides technical evidence that the ELT (Extract, Load, Transform) pipeline, automated as outlined in the code above, successfully pulled the raw files from the Bronze landing zone, performed the cleansing transformations defined in the code provided above (such as null value detection and elimination, whitespace trimming, and duplicates), and ultimately loaded the validated records into the Silver layer. The fact that all tables were "successfully standardized" confirms the data is now structured, reliable and ready for complex business logic and metric calculations used in the next layer of notebooks in the Gold layer.

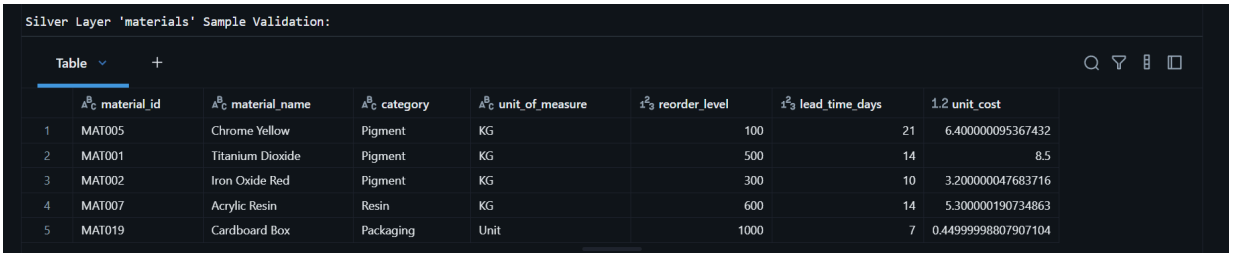


Figure 5.10: Silver Layer 'materials' Sample Validation Output

The figure 5.10 is an example of material discovery for the materials dataset after it has been successfully loaded into the Silver layer of Azure Databricks.The following figure shows an example of material discovery for the materials dataset once it has been successfully loaded into the Silver layer of Azure Databricks. This is a visual example showing that the raw data has been cleaned, parsed and converted to a clean tabular form with enforced schemas.

Key attributes of the material are displayed as output, which are inputs to further calculations downstream for Recoverable Assets (RA) and Magna Carta (MC) risk. For example, the unit_cost column is correctly formatted as a floating-point number, a requirement for the 100% provisions for write-down of expired stock, which is required by the Finance team. Also, the operational measures like reorder_level, lead_time_days are very well defined and easily available for loading into a Fact Constellation Schema in the Gold layer for the final Power BI Dashboard integrations. The consistent formatting of the text in all the categorical columns also confirms that the whitespace trimming and null handling logic in the notebook is well executed.

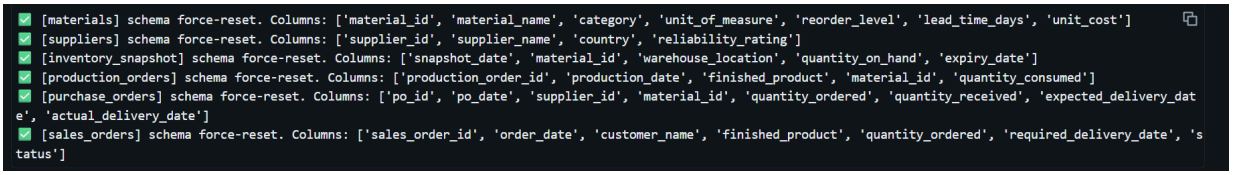


Figure 5.11: Schema Force-Reset Execution Log for Silver Layer Tables

Figure 5.11 shows that the schema enforcement logic was successfully executed when it was run in the Azure Databricks console. The "schema force-reset" messages refer to the script using the overwriteSchema command to replace any data structure that failed to match or was corrupted in the Silver layer. The output clearly specifies the exact column mapping used for all six core datasets (materials, suppliers, inventory_snapshot, production_orders, purchase_orders, and sales_orders), with important data attributes like quantity_on_hand, unit_cost, and material_id being properly organized. This stringent structural validation ensures the integrity of the data at the time they move out of the Bronze layer such that the datasets are exactly structured for the higher-level Recoverable Assets (RA) and Magna Carta (MC) metric calculations in the following Gold layer.

```
['snapshot_date', 'material_id', 'warehouse_location', 'quantity_on_hand', 'expiry_date']
+-------------+-----------+-----------------+---------------+-----------+
|snapshot_date|material_id|warehouse_location|quantity_on_hand|expiry_date|
+-------------+-----------+-----------------+---------------+-----------+
|   2025-01-31|     MAT001|             WH-A|           2000| 2025-12-31|
|   2025-02-28|     MAT001|             WH-A|           2960| 2025-12-31|
|   2025-03-31|     MAT001|             WH-A|           1810| 2025-12-31|
+-------------+-----------+-----------------+---------------+-----------+
only showing top 3 rows
```

Figure 5.12: Data Validation Output for the inventory_snapshot Table

The data validation check for the inventory_snapshot data can be seen in Figure 5.12 below for a typical PySpark dataframe display command. The output of the console confirms the precise structure of the columns that were parsed by the script—snapshot_date, material_id, warehouse_location, quantity_on_hand, and expiry_date. The engineering team can then output top three rows of the dataset to see if the physical stock balances from localised warehouses (e.g. location WH-A) are correctly mapped and formatted. The integrity of these specific data points is important, as the total on hand and expiry date are the most fundamental data points to help determine Recoverable Assets (RA) and Magna Carta (MC) obsolescence risks in the future.

5.5.2 Silver to Gold Notebook

With the Silver layer established, the next stage involved transforming the cleaned and validated data into business-ready Gold tables using Apache Spark on Azure Databricks. The Gold layer serves as the final analytical layer. It contained KPI metrics derived from multiple Silver tables.

Before the transformation logic could be applied, the necessary libraries were imported and the Azure Data Lake Storage connection was configured. The storage account ppgprojectsg was referenced using an account key, and a helper function read_silver() was defined to streamline the reading of Delta tables from the Silver container, as shown in Figure 5.13.

```
# ──────────────────────────────────────────────────────────────
# CELL 1 — Shared Config & Helpers
# ──────────────────────────────────────────────────────────────
from pyspark.sql import functions as F

STORAGE_KEY     = "9v/QDzffL1o0unVZpaLxKGPfCvTVkML3ZPlBJA67IRF4uBmQJ1ajvwb8aY4SGtrh98fbU0XYEVHP+AStK8KBjQ=="
STORAGE_ACCOUNT = "ppgprojectsg"
BASE_URL        = f"abfss://{container}@{STORAGE_ACCOUNT}.dfs.core.windows.net"

def safe_read_silver(table_name):
    return spark.read \
        .format("delta") \
        .option(f"fs.azure.account.key.{STORAGE_ACCOUNT}.dfs.core.windows.net", STORAGE_KEY) \
        .load(f"{BASE_URL.format(container='silver')}/{table_name}")

def write_gold(df, table_name):
    df.write \
        .format("delta") \
        .option(f"fs.azure.account.key.{STORAGE_ACCOUNT}.dfs.core.windows.net", STORAGE_KEY) \
        .mode("overwrite") \
        .option("overwriteSchema", "true") \
        .save(f"{BASE_URL.format(container='gold')}/{table_name}")

print("✅ Config and helpers loaded.")
```

```
✅ Config and helpers loaded.
```

Figure 5.13: Loading Silver Tables

The first Gold table, dim_materials, was built to keep track of all raw materials and their information. This table used the materials Silver table and the inventory_snapshot Silver table. The construction began by calculating the shelf life for each material inside the inventory data. Since this information was not in the main materials list, the days remaining for each item found by subtracting the snapshot date from the expiration date. Then, grouped the data by material_id and picked the maximum remaining days found across all inventory records.

Next, the newly created shelf life data was joined back to the main materials data using a left join on material_id. The final output selected six fields are material_id, material_name, material_category, base_unit_cost, shelf_life_days, and safety_stock_level. During this step, the category field was renamed to material_category, and the unit_cost field was turned into a decimal number named base_unit_cost. The reorder_level field was also converted into a text string and renamed to safety_stock_level because it marks the stock limit that triggers a new order. Finally, any duplicate rows for the same material id were dropped to keep the list completely clean as shown as in Figure 5.14.

```
materials_df         = safe_read_silver("materials")
inventory_df         = safe_read_silver("inventory_snapshot")

# shelf_life_days is not stored on the materials table directly.
# Best available proxy: max remaining shelf life observed across all
# inventory snapshots per material  (max expiry_date - snapshot_date).
shelf_life = inventory_df \
    .withColumn("days_remaining", F.datediff(
        F.to_date(F.col("expiry_date")),
        F.to_date(F.col("snapshot_date"))
    )) \
    .groupBy("material_id") \
    .agg(F.max("days_remaining").alias("shelf_life_days"))

dim_materials = materials_df \
    .join(shelf_life, "material_id", "left") \
    .select(
        F.col("material_id"),
        F.col("material_name"),
        # Silver: 'category'  →  Gold: 'material_category'
        F.col("category").alias("material_category"),
        # Silver: 'unit_cost'  →  Gold: 'base_unit_cost'
        F.col("unit_cost").cast("decimal(18,2)").alias("base_unit_cost"),
        F.col("shelf_life_days").cast("int"),
        # Silver: 'reorder_level' (INT)  →  Gold: 'safety_stock_level' (VARCHAR)
        # reorder_level defines the stock threshold that triggers a replenishment,
        # which is the functional definition of a safety stock level.
        F.col("reorder_level").cast("string").alias("safety_stock_level")
    ) \
    .dropDuplicates(["material_id"])

write_gold(dim_materials, "dim_materials")
```

Figure 5.14 dim_materials Join and Selection Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.15.

```
✅ dim_materials written — 20 rows
```

	material_id	material_name	material_category	base_unit_cost	shelf_life_days	safety_stock_level
0	MAT015	Butanol	Solvent	2.40	455	300
1	MAT005	Chrome Yellow	Pigment	6.40	547	100
2	MAT001	Titanium Dioxide	Pigment	8.50	334	500
3	MAT013	Mineral Spirits	Solvent	1.20	393	600
4	MAT019	Cardboard Box	Packaging	0.45	121	1000

Figure 5.15 dim_materials Successfully Written to Gold

The second Gold table, dim_customer, was built to create a clean list of all unique customers. This table consumed data from the sales_orders. The construction began by pulling customer information from the sales orders records. Because there was no original customer ID column in the source data, a secure SHA256 code was generated from the customer_name field to create a permanent customer_id. A brand new field named industry_type was also added to the table with a default text value of "Unclassified" until more data becomes available. The final

output selected three fields are customer_id, customer_name, and industry_type. The logic then dropped any duplicate rows based on the customer_id to ensure that each customer is only listed one time in the final table shown in Figure 5.16.

```
sales_orders_df = safe_read_silver("sales_orders")

# No native customer_id exists in the silver sales_orders table.
# A stable SHA-256 surrogate key is generated from customer_name so that
# the same customer always gets the same ID across reloads.
# NOTE: fact_sales_orders uses the identical derivation to preserve FK integrity.

# industry_type is not present in any available source table.
# Stored as 'Unclassified' — update once a customer reference table is available.

dim_customer = sales_orders_df \
    .select(
        F.sha2(F.col("customer_name"), 256).alias("customer_id"),
        F.col("customer_name"),
        F.lit("Unclassified").cast("string").alias("industry_type")
    ) \
    .dropDuplicates(["customer_id"])

write_gold(dim_customer, "dim_customer")
print(f"✅ dim_customer written — {dim_customer.count()} rows")
dim_customer.limit(5).toPandas()
```

Figure 5.16 dim_customer Selection and Unique Row Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.17.

✅ dim_customer written — 15 rows

	customer_id	customer_name	industry_type
0	d2a38275de920c75b053614ce23d3b2a56f3f8817a4503...	Maybank Facilities Mgmt	Unclassified
1	231ba638f8208c71ccfd1733ee522b0529941be42b9fda...	Sunway Building Materials	Unclassified
2	de140b6e24f9e888e72200d3b8d0c9da0cb672c9bd2a70...	SP Setia Hardware	Unclassified
3	52aaea0176832c966b49faa2d2a7ea3d42ffe34ed8c981...	BuildRight Hardware	Unclassified
4	74e0bb0271eb0a728fc318f0f0567de9a85be137e9baf3...	UEM Sunrise Bhd	Unclassified

Figure 5.17 dim_customer Successfully Written to Gold

The third Gold table, fact_purchase_orders, was built to track all orders placed with suppliers. This table consumed the purchase_orders. The construction began by selecting the main purchase order fields and creating two new calculated columns. First, a lead_time_days column was calculated by finding the total days between the order date and the delivery date. If the item was already delivered, the actual delivery date was used, but if the order was still open, the logic automatically used the expected delivery date instead.

Second, a true or false flag named po_status was created. This flag turns to True if the amount received is equal to or greater than the amount ordered, meaning the shipment is complete, and False if the order is still open or incomplete. The final output selected eight fields are po_id, order_quantity, received_quantity, lead_time_days, po_status, material_id, supplier_id, and date_id. During this step, quantity_ordered was renamed to order_quantity, quantity_received was renamed to received_quantity, and the original po_date was turned into a clean date format and renamed to date_id to act as a link to the date table as shown in Figure 5.18.

```
purchase_orders_df = safe_read_silver("purchase_orders")

fact_purchase_orders = purchase_orders_df \
    .select(
        F.col("po_id"),
        # Silver: 'quantity_ordered'  →  Gold: 'order_quantity'
        F.col("quantity_ordered").cast("int").alias("order_quantity"),
        # Silver: 'quantity_received'  →  Gold: 'received_quantity'
        F.coalesce(
            F.col("quantity_received").cast("int"),
            F.lit(0)
        ).alias("received_quantity"),
        # Derived: actual days from PO date to delivery
        F.datediff(
            F.to_date(
                F.coalesce(
                    F.when(
                        F.trim(F.col("actual_delivery_date")) == "",
                        None
                    ).otherwise(F.col("actual_delivery_date")),
                    F.col("expected_delivery_date")
                )
            ),
            F.to_date(F.col("po_date"))
        ).alias("lead_time_days"),
        # Derived: fully received = TRUE, still open = FALSE
        F.when(
            F.col("quantity_received").cast("int") >= F.col("quantity_ordered").cast("int"),
            F.lit(True)
        ).otherwise(F.lit(False)).alias("po_status"),
        F.col("material_id"),
        F.col("supplier_id"),
        # Silver: 'po_date'  →  Gold: 'date_id' (FK → dim_date)
        F.to_date(F.col("po_date")).alias("date_id")
    )

write_gold(fact_purchase_orders, "fact_purchase_orders")
```

Figure 5.18 fact_purchase_orders Selection and Calculation Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.19.

✅ fact_purchase_orders written — 40 rows

	po_id	order_quantity	received_quantity	lead_time_days	po_status	material_id	supplier_id	date_id
0	PO-001	2000	2000	21	True	MAT001	SUP006	2025-01-23
1	PO-002	1500	0	14	False	MAT001	SUP006	2025-11-01
2	PO-003	1200	1200	17	True	MAT002	SUP006	2025-02-03
3	PO-004	1500	1500	17	True	MAT002	SUP006	2025-06-28
4	PO-005	800	800	7	True	MAT003	SUP002	2025-01-04

Figure 5.19 fact_purchase_orders Successfully Written to Gold

The fourth Gold table, fact_sales_orders, was built to organize and track customer sales orders. This table combined data from the sales_orders and the production_orders. The construction began by extracting a unique mapping list of finished products and their raw material IDs from the production orders data. This mapping list was then joined with the main sales orders data using a left join on the finished_product field. The final output selected six fields are sales_order_id, order_quantity, fulfillment_status, material_id, date_id, and customer_id. The quantity_ordered field was converted into an integer type and renamed to order_quantity. A true or false column named fulfillment_status was also created, setting the value to True if the order status said "Delivered" and False if it said "Open". The original order_date was turned into a proper date format and renamed to date_id, while a secure SHA256 code was generated from the customer name to build a matching customer_id shown in the Figure 5.20.

```
sales_orders_df_local      = safe_read_silver("sales_orders")
production_orders_df_local = safe_read_silver("production_orders")

product_material_map = production_orders_df_local \
    .select("finished_product", "material_id") \
    .distinct()

fact_sales_orders = sales_orders_df_local \
    .join(product_material_map, "finished_product", "left") \
    .select(
        F.col("sales_order_id"),
        # Silver: 'quantity_ordered'  →  Gold: 'order_quantity'
        F.col("quantity_ordered").cast("int").alias("order_quantity"),
        # Derived from status column: 'Delivered' = TRUE, 'Open' = FALSE
        F.when(
            F.col("status") == "Delivered",
            F.lit(True)
        ).otherwise(F.lit(False)).alias("fulfillment_status"),
        F.col("material_id"),
        # Silver: 'order_date'  →  Gold: 'date_id' (FK → dim_date)
        F.to_date(F.col("order_date")).alias("date_id"),
        # Surrogate key — must match dim_customer.customer_id derivation exactly
        F.sha2(F.col("customer_name"), 256).alias("customer_id")
    )

write_gold(fact_sales_orders, "fact_sales_orders")
```

Figure 5.20 fact_sales_orders Join and Transformation Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.21.

✅ fact_sales_orders written — 192 rows

	sales_order_id	order_quantity	fulfillment_status	material_id	date_id	customer_id
0	SO-001	618	True	MAT006	2025-01-17	52aaea0176832c966b49faa2d2a7ea3d42ffe34ed6c981...
1	SO-002	107	True	MAT006	2025-01-10	6c37f21470c76db56103705cbb31c9260f966da035ae73...
2	SO-003	743	True	MAT008	2025-01-19	fb6504113ac6ef6e711c93e091d3bdc89ed7897b8e9470...
3	SO-004	131	True	MAT017	2025-02-19	f6d29a8989720fab39e6ec43cc1a7e5dc4320e55980b6f...
4	SO-005	468	True	MAT008	2025-02-18	d2a38275de920c75b053614ce23d3b2a56f3f6817a4503...

Figure 5.21 fact_sales_orders Successfully Written to Gold

The fifth Gold table, dim_warehouse, was built to map out all warehouse storage locations. This table used data from the inventory_snapshot. The construction began by selecting the warehouse_location column from the inventory data and dropping all duplicate entries to get a unique list of warehouse codes. The original warehouse_location text was kept as the unique warehouse_id. A new column named location_name was created by stripping the prefix from the code and combining it with the word "Warehouse". The final output selected two fields are warehouse_id and location_name as shown in Figure 5.22.

```
inventory = safe_read_silver("inventory_snapshot")

dim_warehouse = inventory.select(
    F.col("warehouse_location")
).dropDuplicates(["warehouse_location"]) \
.select(
    F.col("warehouse_location").alias("warehouse_id"),
    F.concat(F.lit("Warehouse "), F.regexp_replace(F.col("warehouse_location"), "WH-", "")).alias("location_name"),
)

write_gold(dim_warehouse, "dim_warehouse")
```

Figure 5.22 dim_warehouse ID and Text Formatting Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.23.

✅ dim_warehouse written — 5 rows

	warehouse_id	location_name
0	WH-D	Warehouse D
1	WH-E	Warehouse E
2	WH-B	Warehouse B
3	WH-C	Warehouse C
4	WH-A	Warehouse A

Figure 5.23 dim_warehouse Successfully Written to Gold

The sixth Gold table, dim_date, was built to serve as a calendar tracking master table. This table was constructed entirely inside the notebook by running a local Spark SQL command. The construction began by generating a continuous sequence of daily dates starting from January 1, 2025, all the way through December 31, 2028. This sequence was exploded into individual rows to create a continuous date column named full_date. From this main date, specific calendar fields were extracted using built in time functions. The final output selected six fields are date_id, full_date, year, quarter, month, and day as shown in Figure 5.24.

```
date_spine = spark.sql("""
    SELECT explode(sequence(
        to_date('2025-01-01'),
        to_date('2028-12-31'),
        interval 1 day
    )) AS full_date
""")

dim_date = date_spine.select(
    F.date_format("full_date", "yyyyMMdd").cast("int").alias("date_id"),
    F.col("full_date"),
    F.year("full_date").alias("year"),
    F.quarter("full_date").alias("quarter"),
    F.month("full_date").alias("month"),
    F.dayofmonth("full_date").alias("day")
)

write_gold(dim_date, "dim_date")
```

Figure 5.24 dim_date SQL Generation and Extraction Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.25.

✅ dim_date written — 730 rows

	date_id	full_date	year	quarter	month	day
0	20250101	2025-01-01	2025	1	1	1
1	20250102	2025-01-02	2025	1	1	2
2	20250103	2025-01-03	2025	1	1	3
3	20250104	2025-01-04	2025	1	1	4
4	20250105	2025-01-05	2025	1	1	5

Figure 5.25 dim_date Successfully Written to Gold

The seventh Gold table, dim_suppliers, was built to keep track of all product vendors. This table consumed the supplier. The construction began by selecting the vendor information rows from the silver data source. During this step, several column names were changed to match the final Gold requirements. The reliability_rating column was renamed to vendor_rating, and the country column was renamed to region. The final output selected five fields are supplier_id, supplier_name, vendor_rating, region, and contact_info. After selecting these columns, the logic dropped any duplicate records based on the supplier_id to make sure each supplier appears only once as shown in Figure 5.26.

```
suppliers = safe_read_silver("suppliers")

dim_suppliers = suppliers.select(
    F.col("supplier_id"),
    F.col("supplier_name"),
    F.col("reliability_rating").alias("vendor_rating"),
    F.col("country").alias("region"),
).dropDuplicates(["supplier_id"])

write_gold(dim_suppliers, "dim_suppliers")
print(f"✅ dim_suppliers written — {dim_suppliers.count()} rows")
dim_suppliers.limit(5).toPandas()
```

Figure 5.26 dim_suppliers Column Mapping Logic

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.27.

✅ dim_suppliers written — 10 rows

	supplier_id	supplier_name	vendor_rating	region
0	SUP008	Apex Packaging Sdn Bhd	High	Malaysia
1	SUP006	Global Pigments Ltd	Medium	India
2	SUP002	SinoCoat Materials Co	Medium	China
3	SUP005	Solvent King Pte Ltd	Medium	Singapore
4	SUP010	Polymer Source Ltd	Medium	South Korea

Figure 5.27 dim_suppliers Successfully Written to Gold

The eighth Gold table, fact_inventory_status, was built to provide a detailed view of inventory health and financial stock risk. This table combines data from three different Silver files which are inventory_snapshot, materials, and production_orders. The construction began by converting columns in the inventory, materials, and production orders tables into correct data types like integers, decimals, and dates. Next, two separate rolling consumption metrics were calculated from the production orders data using the current date as a baseline. A rolling 12 month consumption summary and a rolling 9 month consumption summary were built by grouping the filtered production dates by material_id. All of these data pieces were then combined using left joins back to the main inventory data shown in Figure 5.28.

```
inventory   = safe_read_silver("inventory_snapshot")
materials   = safe_read_silver("materials")
prod_orders = safe_read_silver("production_orders")

today = F.current_date()

# Cast types
inventory = inventory \
    .withColumn("quantity_on_hand", F.col("quantity_on_hand").cast("int")) \
    .withColumn("expiry_date",      F.col("expiry_date").cast("date")) \
    .withColumn("snapshot_date",    F.col("snapshot_date").cast("date"))

materials = materials \
    .withColumn("unit_cost",     F.col("unit_cost").cast("double")) \
    .withColumn("reorder_level", F.col("reorder_level").cast("int"))

prod_orders = prod_orders \
    .withColumn("quantity_consumed", F.col("quantity_consumed").cast("int")) \
    .withColumn("production_date",   F.col("production_date").cast("date"))

# Rolling 12m consumption → used for RA flag
rolling_12m = prod_orders.filter(
    F.col("production_date") >= F.add_months(today, -12)
).groupBy("material_id").agg(
    F.sum("quantity_consumed").cast("int").alias("rolling_12m_consumption")
)

# Rolling 9m consumption → used for MC Dead Stock flag
rolling_9m = prod_orders.filter(
    F.col("production_date") >= F.add_months(today, -9)
).groupBy("material_id").agg(
    F.sum("quantity_consumed").cast("int").alias("rolling_9m_consumption")
)

# Join all sources
fact = inventory \
    .join(materials.select("material_id", "unit_cost", "reorder_level"), "material_id", "left") \
    .join(rolling_12m, "material_id", "left") \
    .join(rolling_9m,  "material_id", "left") \
    .withColumn("rolling_12m_consumption",
        F.coalesce(F.col("rolling_12m_consumption"), F.lit(0))
    ) \
    .withColumn("rolling_9m_consumption",
        F.coalesce(F.col("rolling_9m_consumption"), F.lit(0))
    )
```

Figure 5.28 fact_inventory_status Logic

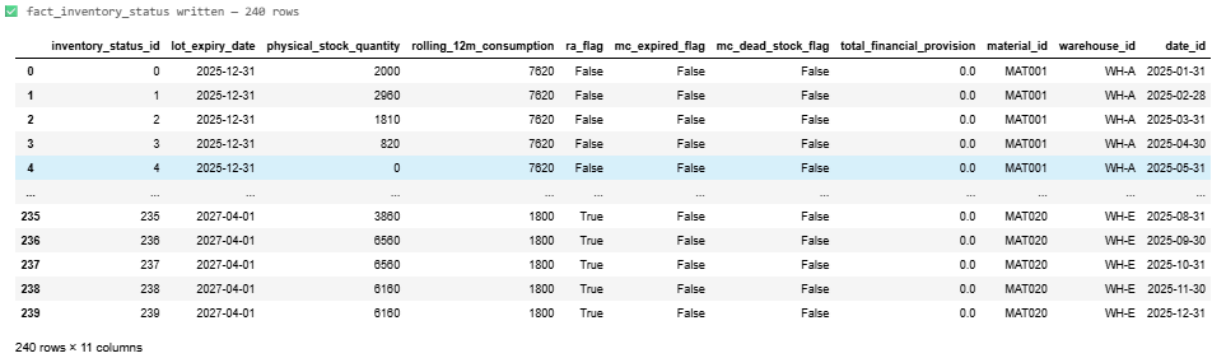
Using this combined information, the code calculated three specific status flags and a final financial metric. The ra_flag was set to True if the physical stock on hand was higher than the rolling 12 month consumption. The mc_expired_flag was set to True if the ra_flag was True and the lot expiration date had already passed. The mc_dead_stock_flag was set to True if the ra_flag was True, there was zero material consumption over the last 9 months, and the material was not already marked as expired.

Finally, the total_financial_provision was calculated by multiplying the physical stock quantity by the unit cost, but only for materials where either the expired flag or dead stock flag was True. The final output selected eleven fields are inventory_status_id, lot_expiry_date, physical_stock_quantity, rolling_12m_consumption, ra_flag, mc_expired_flag, mc_dead_stock_flag, total_financial_provision, material_id, warehouse_id, and date_id. A completely unique row tracking code was generated for the inventory_status_id column as shown in Figure 5.29.

```
fact = fact \
    .withColumn("ra_flag",
        # RA = stock on hand exceeds 12m consumption
        F.when(
            F.col("quantity_on_hand") > F.col("rolling_12m_consumption"),
            True
        ).otherwise(False)
    ) \
    .withColumn("mc_expired_flag",
        # MC Expired = subset of RA where lot expiry date has passed
        F.when(
            (F.col("ra_flag") == True) &
            F.col("expiry_date").isNotNull() &
            (F.col("expiry_date") < today),
            True
        ).otherwise(False)
    ) \
    .withColumn("mc_dead_stock_flag",
        # MC Dead Stock = subset of RA, no consumption 9+ months, no lot expiry
        F.when(
            (F.col("ra_flag") == True) &
            (F.col("rolling_9m_consumption") == 0) &
            (F.col("mc_expired_flag") == False),
            True
        ).otherwise(False)
    ) \
    .withColumn("total_financial_provision",
        # 100% provision for all MC materials (expired or dead stock)
        F.when(
            (F.col("mc_expired_flag") == True) | (F.col("mc_dead_stock_flag") == True),
            F.round(F.col("quantity_on_hand") * F.col("unit_cost"), 2)
        ).otherwise(F.lit(0.0))
    )
```

Figure 5.29 fact_inventory_status Flags Calculations

The completed table was written to the Gold container in Delta format using overwrite mode, as shown in Figure 5.30.



fact_inventory_status written — 240 rows

	inventory_status_id	lot_expiry_date	physical_stock_quantity	rolling_12m_consumption	ra_flag	mc_expired_flag	mc_dead_stock_flag	total_financial_provision	material_id	warehouse_id	date_id
0	0	2025-12-31	2000	7620	False	False	False	0.0	MAT001	WH-A	2025-01-31
1	1	2025-12-31	2960	7620	False	False	False	0.0	MAT001	WH-A	2025-02-28
2	2	2025-12-31	1810	7620	False	False	False	0.0	MAT001	WH-A	2025-03-31
3	3	2025-12-31	820	7620	False	False	False	0.0	MAT001	WH-A	2025-04-30
4	4	2025-12-31	0	7620	False	False	False	0.0	MAT001	WH-A	2025-05-31
...	...	...	...	...	...	...	...	...	...	...	...
235	235	2027-04-01	3860	1800	True	False	False	0.0	MAT020	WH-E	2025-08-31
236	236	2027-04-01	6560	1800	True	False	False	0.0	MAT020	WH-E	2025-09-30
237	237	2027-04-01	6560	1800	True	False	False	0.0	MAT020	WH-E	2025-10-31
238	238	2027-04-01	6160	1800	True	False	False	0.0	MAT020	WH-E	2025-11-30
239	239	2027-04-01	6160	1800	True	False	False	0.0	MAT020	WH-E	2025-12-31

240 rows × 11 columns

Figure 5.30 fact_inventory_status Successfully Written to Gold

5.6 Azure Synapse Analytics Implementation

Azure Synapse Analytics was implemented as the serving layer of the proposed data engineering solution. It's main purpose is to make Gold layer stored in Azure Data Lake Storage Gen2 accessible and queryable downstream to analytical tools such as Power BI. Rather than accessing Delta Lake files directly from storage, Synapse serves the curated datasets as serverless SQL views that users can query with standard SQL statements to access business-ready data.

The dimensional model created in the Silver-to-Gold transformation process is the model implemented in the Synapse. There are 5 dimension datasets (dim_materials, dim_customer, dim_suppliers, dim_warehouse, and dim_date) and 3 fact datasets (fact_inventory_status, fact_purchase_orders, and fact_sales_orders) on the Gold layer. These datasets together constitute the analytical warehouse as a support for the inventory risk analysis, monitoring of supplier performance, assessment of customer impact and reporting of inventory provision.

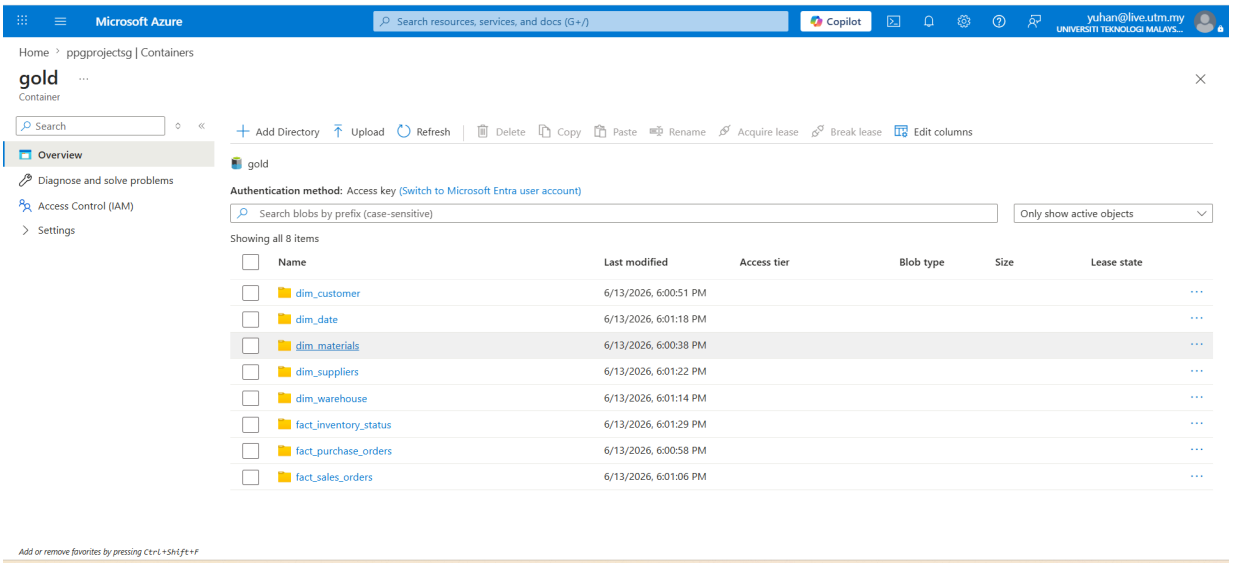


Figure 5.31: Gold layer dimensional and fact datasets generated by the Silver-to-Gold transformation process

Figure 5.31 shows the final datasets stored within the Gold layer of Azure Data Lake Storage Gen2. The Silver-to-Gold Databricks notebook generates five dimension tables and three fact tables based on the Fact Constellation (Galaxy Schema) pattern. The dimension datasets are used for descriptive business context and the fact datasets are used for operational and analytical measurements for inventory risk evaluation and reporting. These Gold layer datasets are the source for Azure Synapse Analytics.

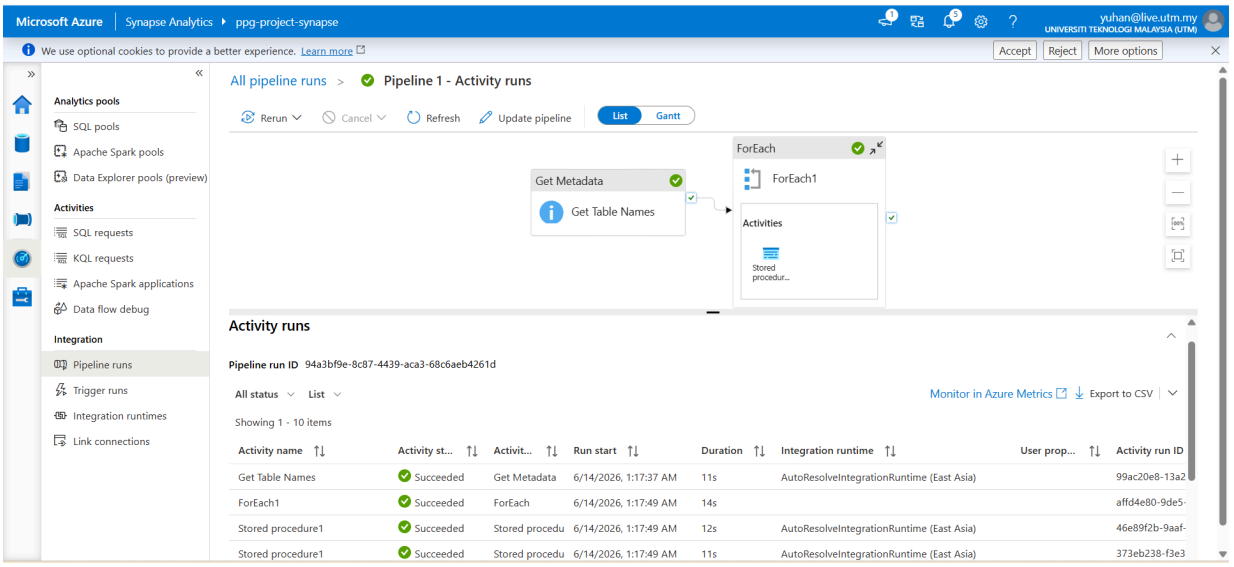


Figure 5.32: Successful execution of the Azure Synapse Analytics automation pipeline

The Synapse pipeline for automating SQL view creation is shown in figure 5.32. The first activity in the pipeline is a Get Metadata activity that fetches all the names of all the datasets in the Gold layer. A ForEach activity then loops through the discovered datasets and runs a stored procedure for each dataset. This automation saves the need for manually creating views and guarantees that newly created Gold layer tables can be exposed through Synapse consistently and efficiently.

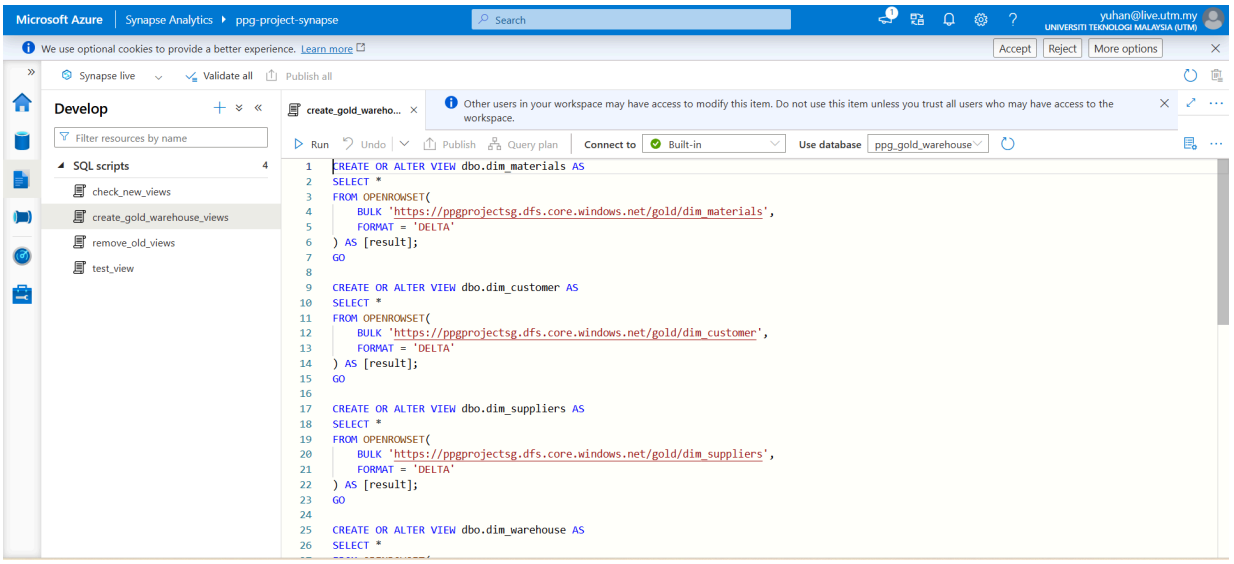


Figure 5.33: SQL script used to create serverless SQL views from Gold layer Delta tables

The SQL script to create the serverless views in the Synapse SQL database is shown in Figure 5.33. The script uses OPENROWSET function with support for Delta format to create virtual views to the Gold layer datasets in Azure Data Lake Storage Gen2. This way, Synapse can directly query the data in Delta Lake without having to duplicate the data in storage and without requiring to change the reporting and analytics tools from the traditional SQL-based tools.

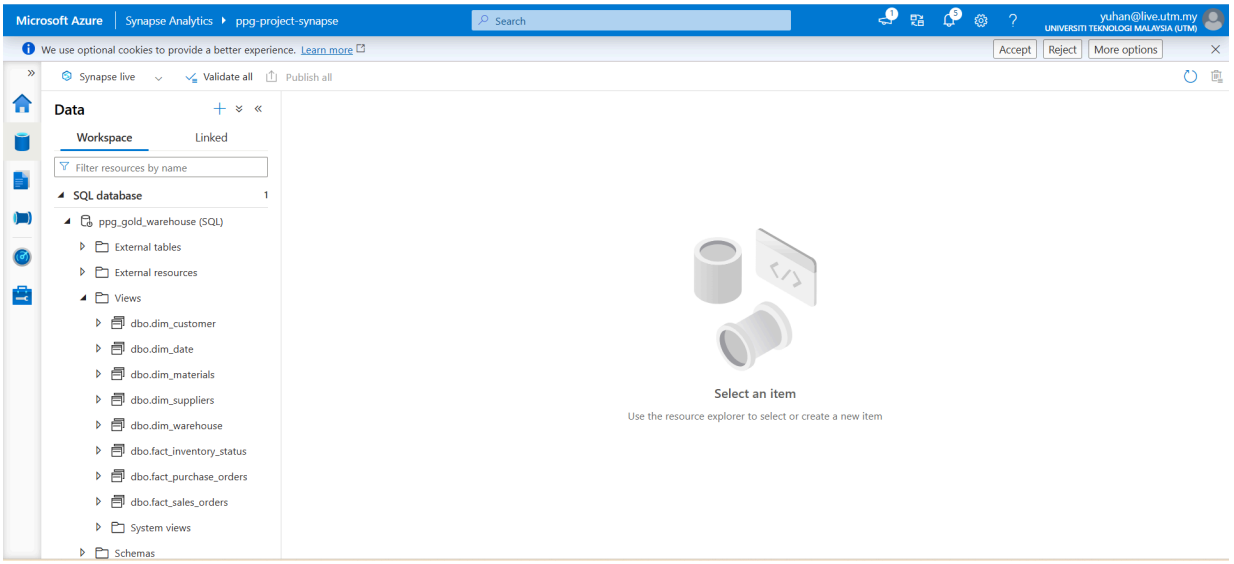


Figure 5.34: Serverless SQL views available in the ppg_gold_warehouse database

Figure 5.34 shows the final serverless SQL views generated within the ppg_gold_warehouse database. The warehouse contains five dimension views and three fact views corresponding to the Gold layer datasets. These views are structured analytical models which can be directly consumed by Power BI for dashboard development and business intelligence reporting. Synapse Analytics abstracts the underlying storage structure, making data easier to access and maintain the scalability and flexibility of a lakehouse architecture.

5.7 Power BI Dashboard Implementation

5.7.1 Data Model Connection to Synapse

In Azure Synapse Analytics, all Gold layer tables were created. Then, this group connects the Microsoft Power BI Desktop to the Synapse Serverless SQL Pool endpoint to retrieve the structured Fact Constellation schema data. Figure 5.19 shows the Fact Constellation schema obtained from the gold layer table from the pipeline.

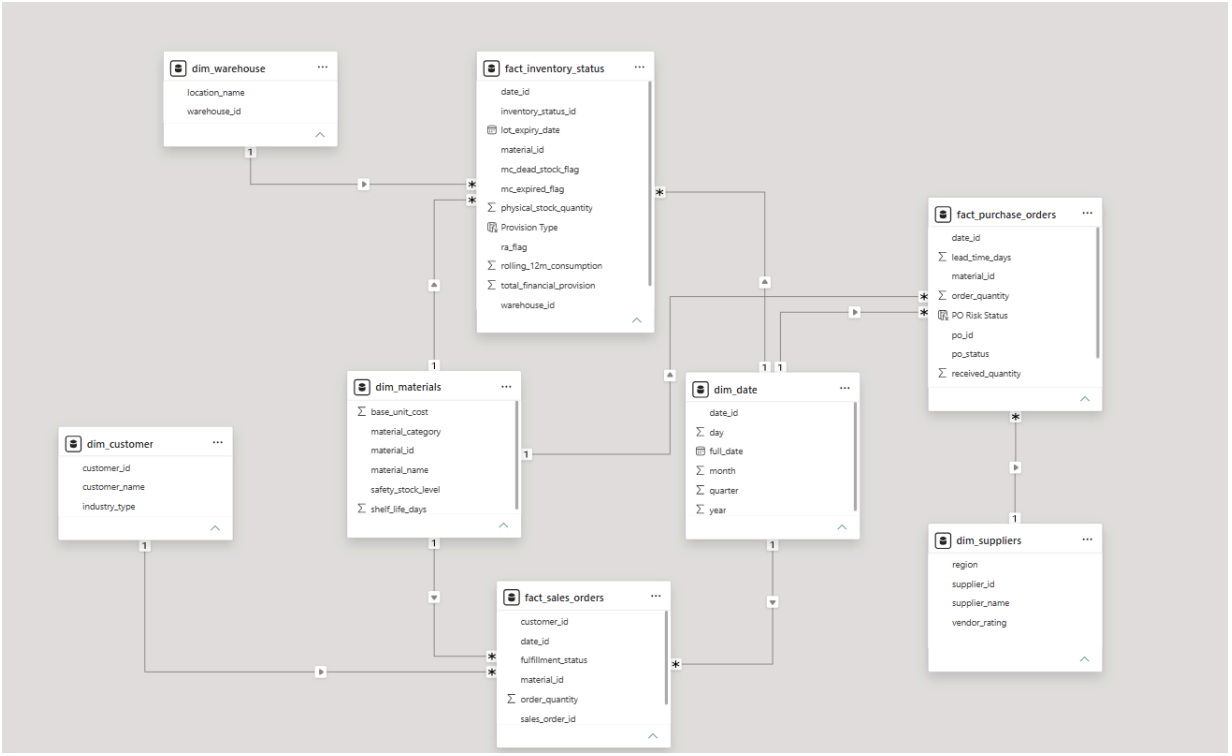


Figure 5.35: Gold Layer Fact Constellation Schema

This fact constellation schema allows the system to analyze separate business processes independently while enabling cross-functional insights, such as mapping material shortages directly to customer orders.

5.7.2 DAX Measures

Custom DAX (Data Analysis Expressions) measures were created to automate the RA and MC business logic calculations and power the KPI scorecards on the dashboard.

```
MC Dead Stock Provision =
CALCULATE(
    SUM(fact_inventory_status[total_financial_provision]),
    fact_inventory_status[mc_dead_stock_flag] = TRUE()
)
```

Figure 5.36: MC Dead Stock Provision

```
Below Reorder Count =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
VAR LatestData =
    CALCULATETABLE(
        VALUES(fact_inventory_status[material_id]),
        fact_inventory_status[date_id] = LatestDate,
        ALL(dim_date),
        fact_inventory_status[is_below_reorder] = TRUE()
    )
RETURN
COUNTROWS(LatestData)
```

Figure 5.37: Below Reorder Count

```
Expiry Date =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    MAX(fact_inventory_status[lot_expiry_date]),
    fact_inventory_status[date_id] = LatestDate,
    ALL(dim_date)
)
```

Figure 5.38: Expiry Date

```
Count Materials Requiring Provision =
CALCULATE(
    COUNTROWS(fact_inventory_status),
    fact_inventory_status[provision_amount] > 0
)
```

Figure 5.39: Count Materials Requiring Provision

```
Inventory Risk Ratio =
DIVIDE(
    SUM(fact_inventory_status[physical_stock_quantity]),
    SUM(fact_inventory_status[rolling_12m_consumption]),
    0
)
```

Figure 5.40: Inventory Risk Ratio

```
is_below_reorder =
VAR LatestDate = MAXX(ALL(fact_inventory_status), fact_inventory_status[date_id])
VAR SafetyLevel = RELATED(dim_materials[safety_stock_level])
RETURN
IF(
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[physical_stock_quantity] < SafetyLevel,
    BLANK()
)
```

Figure 5.41: is_below_rerorder column

```
Latest Stock Quantity =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    SUM(fact_inventory_status[physical_stock_quantity]),
    fact_inventory_status[date_id] = LatestDate,
    ALL(dim_date)
)
```

Figure 5.42: Latest Stock Quantity

```
MC Dead Stock Color =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
VAR IsDeadStock =
    CALCULATE(
        COUNTROWS(fact_inventory_status),
        fact_inventory_status[date_id] = LatestDate,
        fact_inventory_status[mc_dead_stock_flag] = TRUE(),
        ALL(dim_date)
    ) > 0
RETURN
IF(IsDeadStock, "#FF0000", "#D3D3D3")
```

Figure 5.43: MC Dead Stock Color

```
MC Dead Stock Count =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    DISTINCTCOUNT(fact_inventory_status[material_id]),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[mc_dead_stock_flag] = TRUE(),
    ALL(dim_date)
)
```

Figure 5.44: MC Dead Stock Count

```
MC Dead Stock Flag Latest =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    COUNTROWS(fact_inventory_status),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[mc_dead_stock_flag] = TRUE(),
    ALL(dim_date)
) > 0
```

Figure 5.45: MC Dead Stock Flag Latest

```
MC Expired Color =
IF(
    [MC Expired Flag Latest] = TRUE(),
    "#E68EF8",
    "#FFFFFF"
)
```

Figure 5.46: MC Expired Color

```
MC Expired Count =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    DISTINCTCOUNT(fact_inventory_status[material_id]),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[mc_expired_flag] = TRUE(),
    ALL(dim_date)
)
```

Figure 5.47: MC Expired Count

```
MC Expired Flag Latest =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    COUNTROWS(fact_inventory_status),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[mc_expired_flag] = TRUE(),
    ALL(dim_date)
) > 0
```

Figure 5.48: MC Expired Flag Latest

```
MC Expired Provision =
CALCULATE(
    SUM(fact_inventory_status[total_financial_provision]),
    fact_inventory_status[mc_expired_flag] = TRUE()
)
```

Figure 5.49: MC Expired Provision

```
Non RA Count =
VAR LatestDate =
    CALCULATE(MAX(fact_inventory_status[date_id]), ALL(fact_inventory_status))
RETURN
CALCULATE(
    DISTINCTCOUNT(fact_inventory_status[material_id]),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[ra_flag] = FALSE(),
    ALL(dim_date)
)
```

Figure 5.50: Non RA Count

```
RA Color =
IF(
    [RA Flag Latest] = TRUE(),
    "#00B050",
    "#FFFFFF"
)
```

Figure 5.51: RA Color

```
RA Count =
VAR LatestDate =
    CALCULATE(MAX(fact_inventory_status[date_id]), ALL(fact_inventory_status))
RETURN
CALCULATE(
    DISTINCTCOUNT(fact_inventory_status[material_id]),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[ra_flag] = TRUE(),
    ALL(dim_date)
)
```

Figure 5.52: RA Count

```
RA Flag Latest =
VAR LatestDate =
    CALCULATE(
        MAX(fact_inventory_status[date_id]),
        ALL(fact_inventory_status)
    )
RETURN
CALCULATE(
    COUNTROWS(fact_inventory_status),
    fact_inventory_status[date_id] = LatestDate,
    fact_inventory_status[ra_flag] = TRUE(),
    ALL(dim_date)
) > 0
```

Figure 5.53: RA flag latest

```
Safety Stock Level = AVERAGE(dim_materials[safety_stock_level])
```

Figure 5.54: Safety Stock Level

```
Stock Color =
IF(
    SELECTEDVALUE(fact_inventory_status[is_below_reorder]) = TRUE(),
    "#E68F96",
    "#81D7AE"
)
```

Figure 5.55: Stock Color

```
Stockout Risk Count =
CALCULATE(
    COUNTROWS(fact_sales_orders),
    fact_sales_orders[fulfillment_status] = "At Risk"
)
```

Figure 5.56: Stockout Risk Count

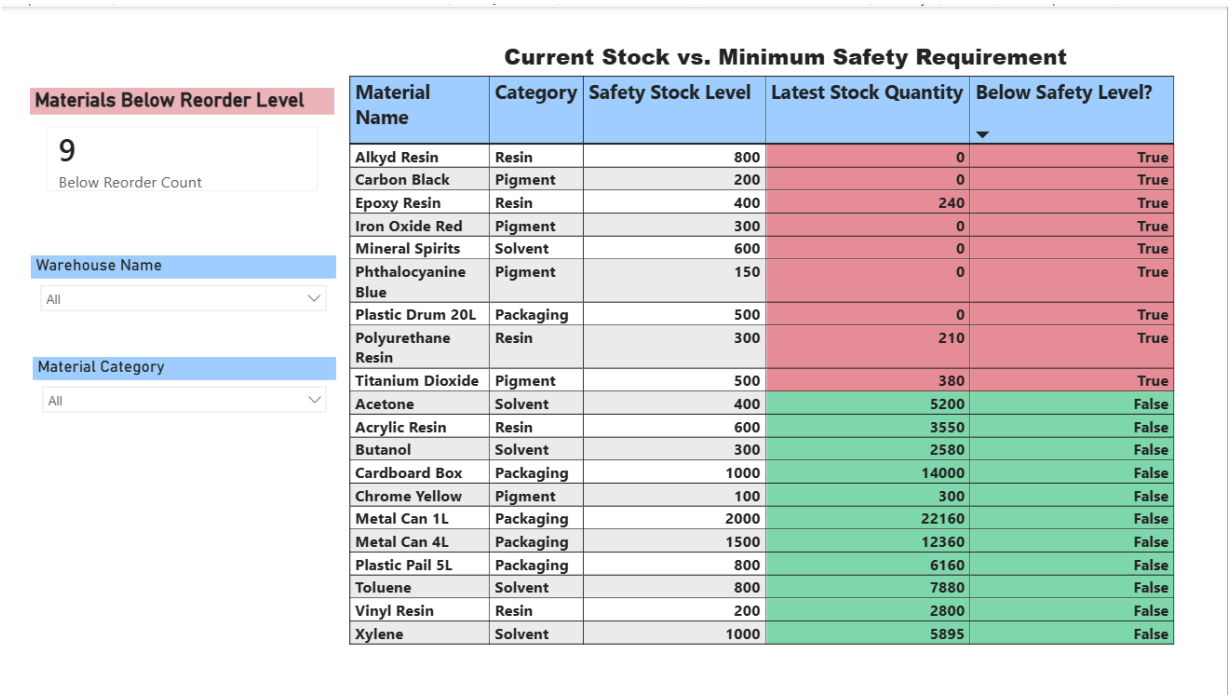
```
1  Test Latest Date =
2  CALCULATE(
3      MAX(fact_inventory_status[date_id]),
4      ALL(fact_inventory_status)
5  )
```

Figure 5.56: Test Latest Date

```
1  Total Provision = SUM(fact_inventory_status[total_financial_provision])
```

Figure 5.57: Total Provision

5.7.3 Dashboard Visualization



Materials Below Reorder Level

9

Below Reorder Count

Warehouse Name

All

Material Category

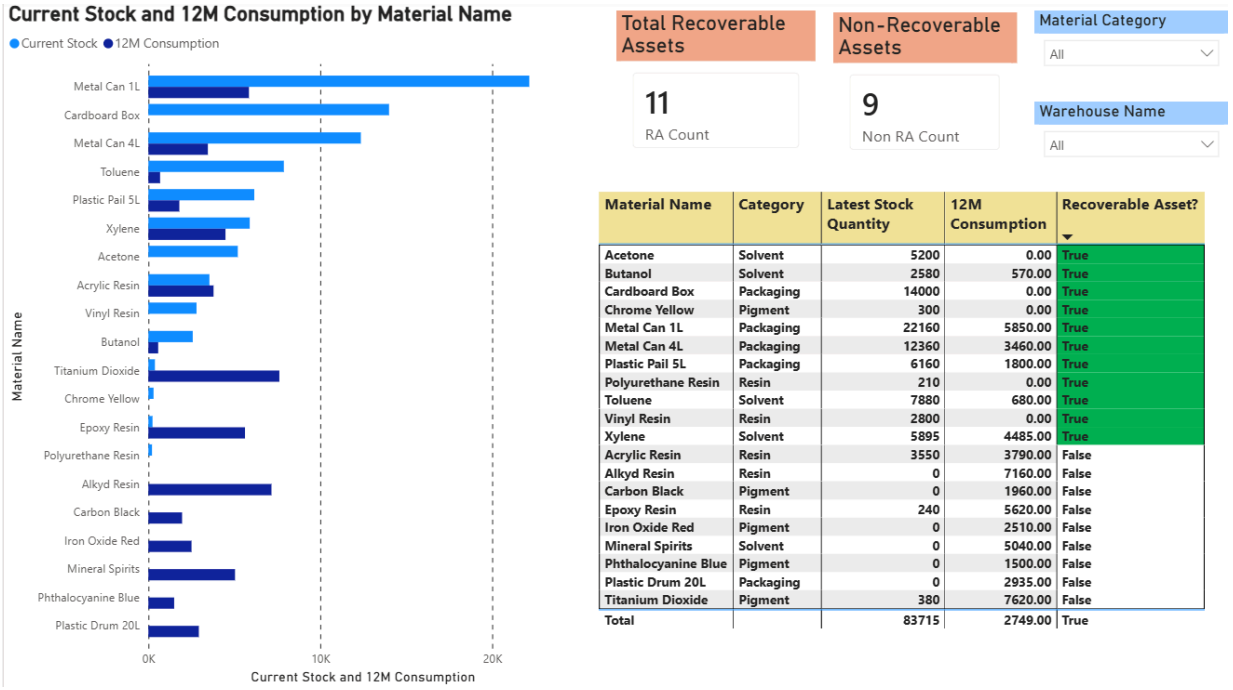
All

Current Stock vs. Minimum Safety Requirement

Material Name	Category	Safety Stock Level	Latest Stock Quantity	Below Safety Level?
Alkyd Resin	Resin	800	0	True
Carbon Black	Pigment	200	0	True
Epoxy Resin	Resin	400	240	True
Iron Oxide Red	Pigment	300	0	True
Mineral Spirits	Solvent	600	0	True
Phthalocyanine Blue	Pigment	150	0	True
Plastic Drum 20L	Packaging	500	0	True
Polyurethane Resin	Resin	300	210	True
Titanium Dioxide	Pigment	500	380	True
Acetone	Solvent	400	5200	False
Acrylic Resin	Resin	600	3550	False
Butanol	Solvent	300	2580	False
Cardboard Box	Packaging	1000	14000	False
Chrome Yellow	Pigment	100	300	False
Metal Can 1L	Packaging	2000	22160	False
Metal Can 4L	Packaging	1500	12360	False
Plastic Pail 5L	Packaging	800	6160	False
Toluene	Solvent	800	7880	False
Vinyl Resin	Resin	200	2800	False
Xylene	Solvent	1000	5895	False

Figure 5.58: Materials Below Reorder Level

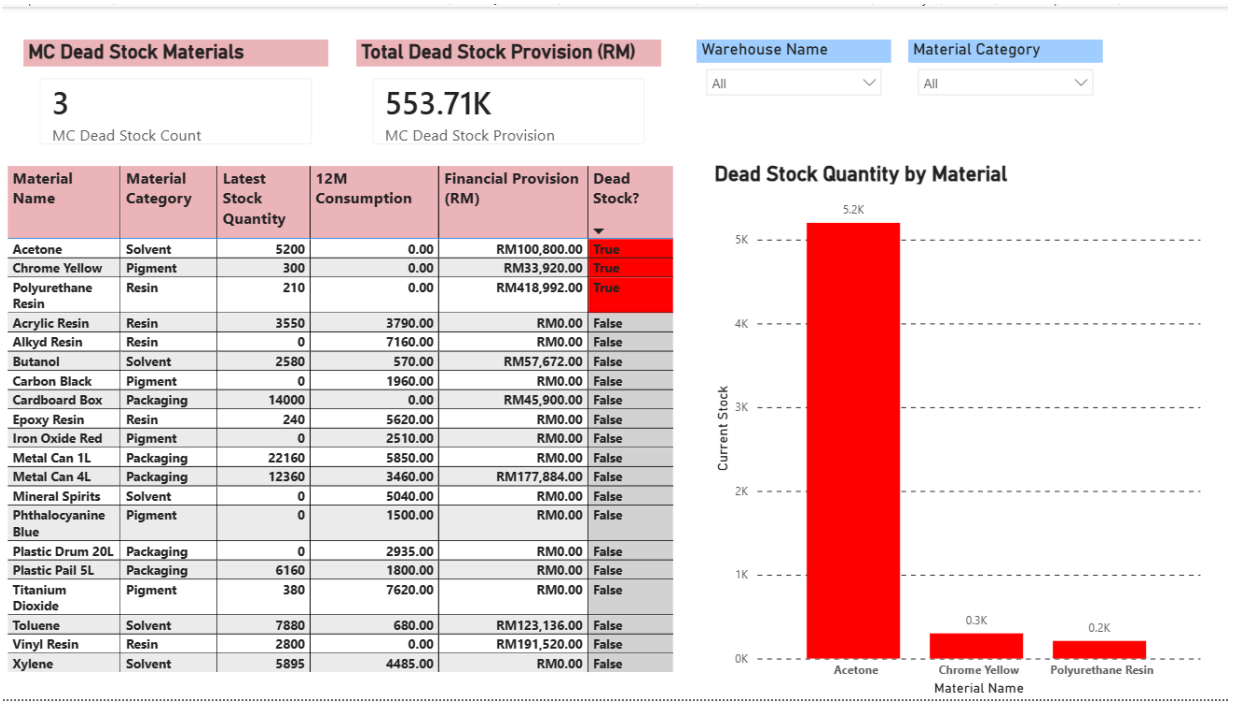
Figure 5.31 shows material that current stock has fallen below the designated safety stock threshold. There are about 9 stocked at the time of reporting. Thus, there needed replenishment action was required.



Material Name	Category	Latest Stock Quantity	12M Consumption	Recoverable Asset?
Acetone	Solvent	5200	0.00	True
Butanol	Solvent	2580	570.00	True
Cardboard Box	Packaging	14000	0.00	True
Chrome Yellow	Pigment	300	0.00	True
Metal Can 1L	Packaging	22160	5850.00	True
Metal Can 4L	Packaging	12360	3460.00	True
Plastic Pail 5L	Packaging	6160	1800.00	True
Polyurethane Resin	Resin	210	0.00	True
Toluene	Solvent	7880	680.00	True
Vinyl Resin	Resin	2800	0.00	True
Xylene	Solvent	5895	4485.00	True
Acrylic Resin	Resin	3550	3790.00	False
Alkyd Resin	Resin	0	7160.00	False
Carbon Black	Pigment	0	1960.00	False
Epoxy Resin	Resin	240	5620.00	False
Iron Oxide Red	Pigment	0	2510.00	False
Mineral Spirits	Solvent	0	5040.00	False
Phthalocyanine Blue	Pigment	0	1500.00	False
Plastic Drum 20L	Packaging	0	2935.00	False
Titanium Dioxide	Pigment	380	7620.00	False
Total		83715	2749.00	True

Figure 5.59: Recoverable Assets

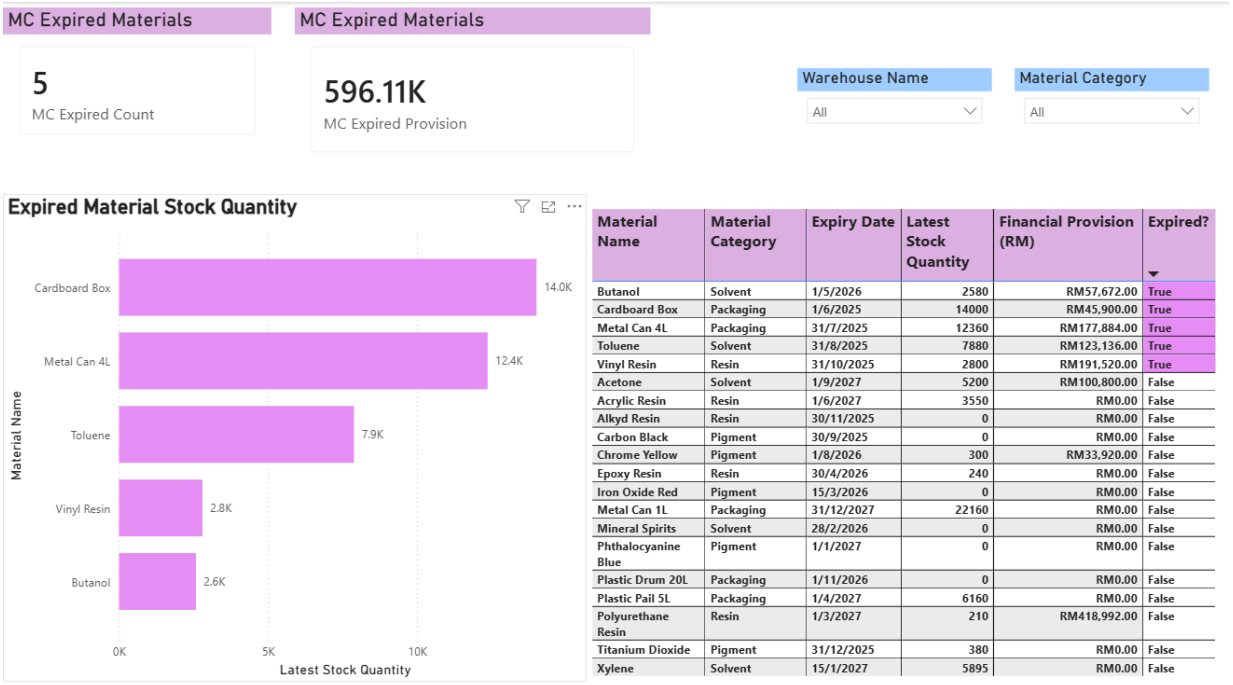
Figure 5.32 shows the material flagged as recoverable assets. Not only that, for non-recoverable assets also shown.



Material Name	Material Category	Latest Stock Quantity	12M Consumption	Financial Provision (RM)	Dead Stock?
Acetone	Solvent	5200	0.00	RM100,800.00	True
Chrome Yellow	Pigment	300	0.00	RM33,920.00	True
Polyurethane Resin	Resin	210	0.00	RM418,992.00	True
Acrylic Resin	Resin	3550	3790.00	RM0.00	False
Alkyd Resin	Resin	0	7160.00	RM0.00	False
Butanol	Solvent	2580	570.00	RM57,672.00	False
Carbon Black	Pigment	0	1960.00	RM0.00	False
Cardboard Box	Packaging	14000	0.00	RM45,900.00	False
Epoxy Resin	Resin	240	5620.00	RM0.00	False
Iron Oxide Red	Pigment	0	2510.00	RM0.00	False
Metal Can 1L	Packaging	22160	5850.00	RM0.00	False
Metal Can 4L	Packaging	12360	3460.00	RM177,884.00	False
Mineral Spirits	Solvent	0	5040.00	RM0.00	False
Phthalocyanine Blue	Pigment	0	1500.00	RM0.00	False
Plastic Drum 20L	Packaging	0	2935.00	RM0.00	False
Plastic Pail 5L	Packaging	6160	1800.00	RM0.00	False
Titanium Dioxide	Pigment	380	7620.00	RM0.00	False
Toluene	Solvent	7880	680.00	RM123,136.00	False
Vinyl Resin	Resin	2800	0.00	RM191,520.00	False
Xylene	Solvent	5895	4485.00	RM0.00	False

Figure 5.60: Magna Carta Dead Stock

Figure 5.33 shows the materials with zero consumption for 9 or more months totaling RM553.71K in at risk value. Even with only 3 materials, total provision is high.



Material Name	Material Category	Expiry Date	Latest Stock Quantity	Financial Provision (RM)	Expired?
Butanol	Solvent	1/5/2026	2580	RM57,672.00	True
Cardboard Box	Packaging	1/6/2025	14000	RM45,900.00	True
Metal Can 4L	Packaging	31/7/2025	12360	RM177,884.00	True
Toluene	Solvent	31/8/2025	7880	RM123,136.00	True
Vinyl Resin	Resin	31/10/2025	2800	RM191,520.00	True
Acetone	Solvent	1/9/2027	5200	RM100,800.00	False
Acrylic Resin	Resin	1/6/2027	3550	RM0.00	False
Alkyd Resin	Resin	30/11/2025	0	RM0.00	False
Carbon Black	Pigment	30/9/2025	0	RM0.00	False
Chrome Yellow	Pigment	1/8/2026	300	RM33,920.00	False
Epoxy Resin	Resin	30/4/2026	240	RM0.00	False
Iron Oxide Red	Pigment	15/3/2026	0	RM0.00	False
Metal Can 1L	Packaging	31/12/2027	22160	RM0.00	False
Mineral Spirits	Solvent	28/2/2026	0	RM0.00	False
Phthalocyanine Blue	Pigment	1/1/2027	0	RM0.00	False
Plastic Drum 20L	Packaging	1/11/2026	0	RM0.00	False
Plastic Pail 5L	Packaging	1/4/2027	6160	RM0.00	False
Polyurethane Resin	Resin	1/3/2027	210	RM418,992.00	False
Titanium Dioxide	Pigment	31/12/2025	380	RM0.00	False
Xylene	Solvent	15/1/2027	5895	RM0.00	False

Figure 5.61: Magna Carta Expired Stock

Figure 5.34 shows materials that have passed their certified lot expiry dates, requiring a full 100% financial write-down totalling RM 596.11K. The highest-value expired materials were Vinyl Resin (RM 191,520), Metal Can 4L (RM 177,884), and Toluene (RM 123,136).

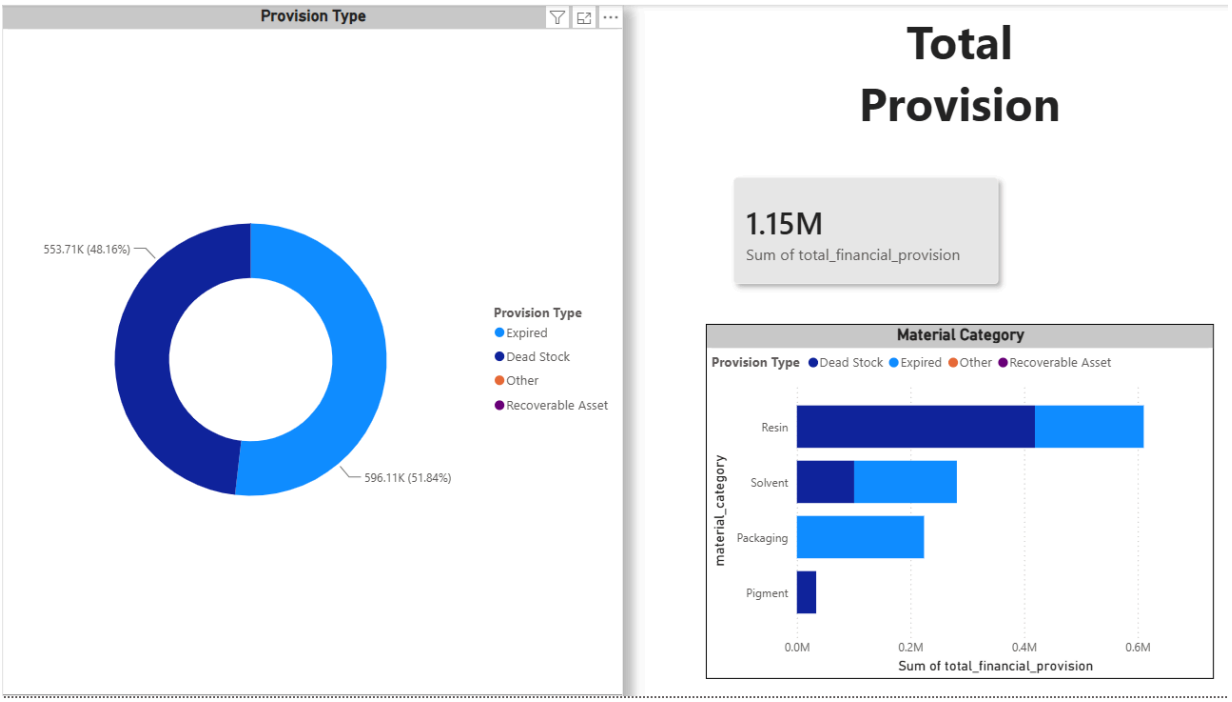


Figure 5.62: MC Provision Required

Figure 5.35 shows the total financial provision liability across all flagged MC materials. materials. 4 materials category identified as requiring financial provision. Resin and Solvent categories carried the highest combined provision burden.

material_name	Sum of order_quantity	Sum of received_quantity	Shortfall	supplier_name	PO Risk St
Acetone	3600	3600	0	Solvent King Pte Ltd	Fulfilled
Acrylic Resin	7200	7200	0	Resin Tech Asia	Fulfilled
Alkyd Resin	6400	6400	0	Resin Tech Asia	Fulfilled
Butanol	1200	960	240	Pacific Solvents Inc	At Risk
Butanol	1500	1500	0	Pacific Solvents Inc	Fulfilled
Carbon Black	2000	2000	0	SinoCoat Materials Co	Fulfilled
Cardboard Box	5000	4000	1000	Apex Packaging Sdn Bhd	At Risk
Cardboard Box	6000	6000	0	Apex Packaging Sdn Bhd	Fulfilled
Chrome Yellow	600	600	0	SinoCoat Materials Co	Fulfilled
Epoxy Resin	2400	2400	0	Polymer Source Ltd	Fulfilled
Iron Oxide Red	2700	2700	0	Global Pigments Ltd	Fulfilled
Metal Can 1L	24000	24000	0	PackPro Solutions	Fulfilled
Metal Can 4L	12000	12000	0	PackPro Solutions	Fulfilled
Mineral Spirits	4200	4200	0	Pacific Solvents Inc	Fulfilled
Phthalocyanine Blue	1350	1350	0	EuroChem Imports	Fulfilled
Plastic Drum 20L	3000	3000	0	Apex Packaging Sdn Bhd	Fulfilled
Plastic Pail 5L	3200	2560	640	Apex Packaging Sdn Bhd	At Risk
Total	**117550**	**110330**	**7220**		

Figure 5.63: Stockout Risk and Affected Customer

Figure 5.36 shows the comparison of current stock on hand against open sales demand. The total material that is at risk is 6. But customer affected shown at Figure 5.36 above is 15 because all the customer affected to material, but only few that affected to 'at risk' condition.

5.8 Testing

System testing was conducted to verify that the implemented inventory risk management solution satisfies the functional requirements identified during the analysis phase. The testing activities evaluated the ability of the system to ingest operational datasets, process inventory risk logic, generate analytical datasets, provide business-ready information through the analytical warehouse, and support decision-making through dashboard reporting. The testing approach was aligned with the implemented Medallion Architecture and covered the Bronze, Silver, Gold, Synapse Analytics, and dashboard layers.

5.8.1 Bronze Layer Testing

Bronze Layer testing was performed to verify that all operational datasets required for Recoverable Assets (RA) and Magna Carta (MC) inventory analysis were successfully ingested into the cloud environment.

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass / Fail
BT01	Verify that all operational datasets required for inventory risk analysis are successfully ingested into the Bronze layer.	The Azure Data Factory ingestion pipeline was executed and the contents of the Bronze container were inspected after completion.	All six source datasets required for inventory analysis should be available in the Bronze layer.	The materials, inventory_snapshot, purchase_orders, production_orders, suppliers, and sales_orders datasets were successfully ingested and	Pass

				stored within the Bronze layer.	
BT02	Verify that the ingestion process preserves the original operational data required for subsequent transformation and auditing purposes.	The ingested datasets were reviewed within Azure Data Lake Storage Gen2.	Source datasets should remain in their original format without modification.	All datasets were preserved in their original form and remained available for downstream processing and auditing activities.	Pass
BT03	Verify that all ingested datasets in the Bronze layer are stored in the correct target file format (Parquet) to ensure compatibility with downstream pipelines.	Inspect the file extensions and metadata of the ingested datasets within the Bronze container in Azure Data Lake Storage Gen2.	All datasets must be saved using the .parquet file format to optimize read performance for the Databricks processing engine.	All six datasets were verified to be stored as .parquet files in the correct parquet/ directory structure within the Bronze container.	Pass

Table 5.1 Bronze Layer Testing

5.8.2 Silver Layer Testing

Silver Layer testing was conducted to verify that the inventory datasets were successfully cleansed, standardized, and prepared for inventory risk evaluation.

Test ID	**Test Description**	**Input / Steps**	**Expected Result**	**Actual Result**	**Pass / Fail**
ST01	Verify that operational datasets can be transformed into standardized	The Bronze-to-Silver Databricks notebook was executed and the	The transformation process should successfully generate	The notebook completed successfully and generated standardized	Pass

	datasets suitable for inventory risk analysis.	resulting Silver datasets were reviewed.	cleansed and standardized datasets.	datasets suitable for downstream analytical processing.	
ST02	Verify that data quality issues do not prevent inventory analysis from being performed.	The generated Silver datasets were inspected following notebook execution.	The transformed datasets should be consistent and suitable for analytical processing.	The datasets were successfully prepared for inventory risk evaluation and no critical data quality issues were identified.	Pass

Table 5.2 Silver Layer Testing

5.8.3 Gold Layer Testing

Gold Layer testing was performed to verify that inventory risk indicators and analytical datasets required for business decision-making were successfully generated.

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass / Fail
GT01	Verify that inventory master data, supplier information, warehouse information, customer information, and calendar attributes are successfully transformed into analytical dimension datasets.	The contents of the Gold layer were inspected after execution of the Silver-to-Gold notebook.	Five business dimension datasets should be generated.	The dim_materials, dim_customer, dim_suppliers, dim_warehouse, and dim_date datasets were successfully generated.	Pass
GT02	Verify that inventory status,	The Gold layer datasets were	Analytical fact datasets should	The fact_inventory_s	Pass

	purchase order, and sales order information are transformed into analytical fact datasets for inventory monitoring and operational analysis.	reviewed following notebook execution.	be generated for inventory monitoring and business reporting.	tatus, fact_purchase_orders, and fact_sales_orders datasets were successfully generated.	
GT03	Verify that the analytical datasets required for Recoverable Assets (RA) and Magna Carta (MC) inventory analysis are successfully created.	The generated Gold layer datasets were compared against the dimensional model defined in Chapter 4.	The Gold layer should contain all datasets required to support inventory risk analysis.	All required analytical datasets were successfully generated according to the proposed Galaxy Schema design.	Pass

Table 5.3 Gold Layer Testing

5.8.4 Azure Synapse Analytics Testing

Azure Synapse Analytics testing was conducted to verify that business-ready datasets could be accessed through the analytical warehouse environment for reporting and decision support purposes.

Test ID	**Test Description**	**Input / Steps**	**Expected Result**	**Actual Result**	**Pass / Fail**
AS01	Verify that all dimension and fact datasets generated in the Gold layer are successfully exposed as Synapse SQL views.	Execute the SQL view creation script and inspect the ppg_gold_warehouse database.	Five dimension views and three fact views should be created successfully.	The dim_materials, dim_customer, dim_suppliers, dim_warehouse, dim_date, fact_inventory_status, fact_purchase_o	Pass

				rders, and fact_sales_orders views were successfully created.	
AS02	Verify that inventory risk information stored in the fact_inventory_status view can be accessed through Synapse Analytics.	Execute the query SELECT TOP 10 * FROM dbo.fact_inventory_status;	Inventory records containing risk-related information should be returned successfully.	The query executed successfully and returned inventory status records from the analytical warehouse.	Pass
AS03	Verify that dimension datasets required for inventory analysis are accessible through the analytical warehouse.	Execute queries against dim_materials, dim_suppliers, dim_warehouse, dim_customer, and dim_date views.	Dimension records should be returned successfully and be available for analytical reporting.	All dimension views returned valid records and were successfully accessed through Synapse Analytics.	Pass
AS04	Verify that the analytical warehouse provides the datasets required by Power BI dashboards for inventory risk monitoring.	Connect Power BI to the Synapse warehouse views and refresh the dataset.	Power BI should successfully retrieve data from all required warehouse views.	The dashboard successfully retrieved data from Synapse and displayed inventory risk metrics without errors.	Pass

Table 5.4 Azure Synapse Analytics Testing

5.8.5 User Acceptance Testing

User Acceptance Testing (UAT) was conducted to verify that the dashboard satisfies the inventory monitoring and decision-support requirements identified by PPG.

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass / Fail
UAT01	Verify that stakeholders can identify Recoverable Assets (RA) inventory through the dashboard.	The Recoverable Assets dashboard page was reviewed.	Inventory classified as Recoverable Assets should be clearly displayed for analysis and monitoring.	The dashboard successfully displayed inventory classified as Recoverable Assets.	Pass
UAT02	Verify that stakeholders can identify Magna Carta inventory requiring financial attention.	The Magna Carta dashboard pages were reviewed.	Dead stock and expired inventory should be clearly identified through dashboard visualizations.	The dashboard successfully displayed Magna Carta inventory and associated risk indicators.	Pass
UAT03	Verify that stakeholders can monitor inventory risk indicators and operational impacts through the dashboard environment.	The complete dashboard solution was reviewed using the identified business scenarios.	Inventory risks, operational impacts, and analytical metrics should be available for decision-makin g.	The dashboard successfully presented the required analytical information and supported inventory risk monitoring activities.	Pass
UAT04	Verify that stakeholders can monitor materials that below reorder level	The complete dashboard on materials quantity	Materials below reorder level should be clearly display	The dashboard successfully display the reorder level which is below.	

Table 5.5 User Acceptance Testing

5.8.6 Testing Summary

The testing activities confirmed that the proposed inventory risk management solution successfully supports operational data integration, inventory risk evaluation, analytical data generation, warehouse accessibility, and business intelligence reporting. All test cases were

completed successfully, demonstrating that the implemented solution satisfies the requirements for Recoverable Assets identification, Magna Carta detection, and inventory risk monitoring within PPG Industries Malaysia.

5.9 Summary

The technical execution and physical deployment of the PPG Inventory Risk Management analytics pipeline successfully leveraged the Medallion Architecture within the Microsoft Azure ecosystem. By implementing this modern data lakehouse approach, the engineering team transitioned a fragmented, manual data process into a fully automated, scalable cloud solution.

Throughout the implementation phase, each tier of the architecture was systematically constructed: Azure Data Factory reliably orchestrated the ingestion of raw operational files into the Bronze layer; Azure Databricks provided the computational power to cleanse and apply complex business logic in the Silver layer; and the data was subsequently modeled into a highly optimized Fact Constellation Schema within the Gold layer. Finally, the integration of Azure Synapse Analytics established a high-performance serving layer, seamlessly bridging the curated data with Microsoft Power BI.

Ultimately, the deployment of this end-to-end infrastructure ensures a secure, reliable, and automated flow of data. It successfully fulfills the project's technical objectives by providing the business intelligence layer with the precise, high-quality metrics required to proactively monitor stock health, calculate financial provisions, and mitigate the risks associated with Recoverable Assets (RA) and Magna Carta (MC) dead stock.

REFERENCES

Bergh, C. (2024). *The race for data quality in a medallion architecture*. DataKitchen. https://datakitchen.io/the-race-for-data-quality-in-a-medallion-architecture/

BPCS. (2025, January 23). *Azure-powered inventory control tower: Revolutionizing supply chain management*. https://bpcs.com/case-studies/azure-powered-inventory-control-tower-revolutionizing-supply-chain-management

Carrilho, F. (2025, February 13). *Designing and Implementing a Metadata-driven Modern Data Warehouse: Automating ELT Processes through Metadata-Driven Pipelines*. Run.unl.pt. https://run.unl.pt/entities/publication/934ff758-4fdb-4e80-8652-786b70b1b723

Databricks. (2022). *What is a medallion lakehouse architecture?* https://www.databricks.com/glossary/medallion-architecture

Erl, T., Puttini, R., & Mahmood, Z. (2013). *Cloud computing: Concepts, technology & architecture*. Pearson Education.

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling*. Wiley.

Microsoft. (2023). *Data Analysis Expressions (DAX) overview*. Microsoft Learn. https://learn.microsoft.com/en-us/dax/dax-overview

Nalluru, S. K. (2024). Cloud-native ETL: Integrating Databricks and Azure Data Factory for scalable data processing in enterprise environments. *International Journal for Multidisciplinary Research (IJFMR)*, *6*(6). https://www.ijfmr.com/papers/2024/6/29886.pdf

Reis, J., & Housley, M. (2022). *Fundamentals of data engineering*. O'Reilly Media.

Silver, E. A., Pyke, D. F., & Thomas, D. J. (2016). *Inventory and production management in supply chains*. CRC Press. https://doi.org/10.1201/9781315374406

Rucco, C., Longo, A., & Saad, M. (2024). Optimizing Data Ingestion for Big Data: A Cloud-Based Design Pattern Approach. *2021 IEEE International Conference on Big Data (Big Data)*, 3556–3561. https://doi.org/10.1109/bigdata62323.2024.10825970

Simran, S. (2025). Waterfall model in software engineering. Bdtask Blog. https://www.bdtask.com/blog/waterfall-model-in-software-engineering

Staff, C. (2026b, March 14). *What is ERP? Understanding Enterprise Resource Planning*. Coursera. https://www.coursera.org/articles/what-is-erp

WNS. (2025, April 8). *Analytics-driven inventory modeling delivers significant cost benefits*. https://www.wns.com/perspectives/case-studies/engineering-smarter-inventory-management-with-analytics-led-optimization-for-manufacturing-major